Concept Graph Attention: Sequence Modeling over Fixed Concept Memory

Anonymous Author(s)
Affiliation
Address
email

Abstract

Sequence memory is an information constraint: before a predictor can use a distinction, its runtime state must make that distinction observable. This paper develops Concept Graph Attention (CGA) from that first-principles view. We formalize a runtime state as an information partition, separate *repair* updates that change payloads inside a fixed partition from *refinement* updates that change the partition itself, and prove a repair-only obstruction: if the current partition merges cases whose refined Bayes actions differ, no amount of local repair, caching, or optimizer sophistication can remove the resulting linear regret against the refined oracle.

The theory identifies which distinctions are worth adding. A candidate bit has value only through risk reduction, not through geometric novelty or frequency. Under log loss, the population value of a binary distinction is $I(Y; U \mid \Pi)$; under squared loss, it is the conditional mean-separation energy induced by the split. We then give a guarded empirical refinement rule that accepts structural growth only when a held-out, cost-penalized refinement gain beats the best repair gain by a finite-sample margin. The full appendix gives the measure-theoretic join form, finite-class oracle inequalities, graph data-structure bounds, and projection-memory/CPM bridge proofs.

CGA is the sequence-modeling specialization of this refine-or-repair program. It keeps attention's continuous readout but changes the memory basis from prefix tokens to a bounded bank of concept identities. Each concept has a fixed address and a runtime value memory. Tokens read from the concept bank with attention-style softmax weights and write back with gated continuous updates and trainable retention. During training, gradients flow through the read, write, gate, and retention parameters; during inference, model weights stay fixed while the concept value memory is updated by the same forward rule. Empirically, we evaluate CGA through state-equalized MQAR and matched language-model pretraining protocols against modern efficient sequence mixers.

1 Introduction

A sequence model is not only an optimizer; it is an information structure. At any prediction time, its runtime state decides which past distinctions are still observable. If two histories induce the same state, every downstream predictor that factors through that state must treat them the same, no matter how much compute is spent repairing the local payload. This is the first-principles obstruction behind the paper: some errors are payload errors, and some errors are information errors. Payload errors can be repaired inside the current representation. Information errors require a refinement of the state space.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

This distinction is easy to hide in neural notation. A cache update, a recurrent update, an adapter step, and a test-time optimizer all change numbers. The question is whether they also change the quotient of histories that the predictor is allowed to condition on. We therefore start from a decision-theoretic model. A runtime representation induces a partition Π of cases. A repair update changes the action or memory attached to each atom of Π while preserving the partition. A refinement joins Π with a new prediction-time observable distinction U. The central decomposition is

$$R_L(f) - R_L(\Pi^\star) = \underbrace{R_L(f) - R_L(\Pi)}_{\text{repair gap}} + \underbrace{R_L(\Pi) - R_L(\Pi^\star)}_{\text{refinement gap}},$$

for any refined comparator $\Pi^\star$. The repair gap is the residual error of the current payload at fixed information. The refinement gap is the value of changing what the state can distinguish. If the second term is positive, a repair-only learner pays that gap every round in expectation.

The theory also says which distinctions matter. A new bit is not valuable because it is balanced, surprising, or geometrically clean. It is valuable when it changes the Bayes action. For log loss, the value of splitting Π by U is exactly the conditional mutual information $I(Y; U \mid \Pi)$. For squared loss, it is the conditional mean-separation energy created by the split. These identities give the paper's guiding rule: propose distinctions broadly, but accept them only when a held-out, cost-penalized refinement gain beats the best repair gain by a statistical guard.

Concept Graph Attention (CGA) is the sequence-modeling implementation of this refine-or-repair view. Fully general DGM-style memory can create new concepts and edges online. For large-scale language modeling, that raises systems and evaluation questions that obscure the core architectural test. CGA therefore uses a fixed concept budget: each layer has C concept identities, and the online state is the value memory attached to those identities. The fixed bank is not a retreat from the theory; it is a controlled capacity model. The question becomes whether a bounded set of runtime concepts can serve as the memory basis for attention, replacing the token-indexed table while preserving a forward-time learning mechanism.

CGA keeps the familiar attention read,

$$\mathrm{read}(q_t) = \sum_{j=1}^{C} \mathrm{softmax}_j(s(q_t, c_{tj}))\, m_{tj},$$

but changes the objects being read. Instead of storing one key-value entry per previous token, each layer stores value contents over fixed concept slots. Tokens read from these slots with attention-style weights and write into them with gated continuous updates and trainable retention. Thus the layer is attention-like in its readout but recurrent-memory-like in its state:

$$\begin{aligned} \text{Transformer: tokens} &\to \text{attention over token memory,} \\ \text{CGA: tokens} &\to \text{fixed concept memory} \to \text{attention over concepts.} \end{aligned}$$

This gives a different answer to the question that also motivates test-time-training sequence layers [Sun et al., 2025]. TTT layers make the hidden state a small model and update that model by a self-supervised learning step on the test sequence. CGA instead makes the hidden state a concept-indexed memory and updates that memory by a forward-only write rule:

$$\begin{aligned} &\text{TTT:} \quad \text{test-time learning of hidden model weights,} \\ &\text{CGA:} \quad \text{test-time learning of concept-memory contents.} \end{aligned}$$

CGA does not update global model parameters during inference and does not run an inference-time backward pass. Its fast adaptation is the causal update of concept memory.

This paper makes four contributions.

- **First-principles theory.** We formalize runtime memory as an information partition and prove that repair-only learners suffer linear regret whenever the missing distinction has positive Bayes-risk value.
- **Value of distinction.** We characterize useful refinements by consequence value, including the log-loss identity $V_{\log}(U \mid \Pi) = I(Y; U \mid \Pi)$, the squared-loss variance identity, and a guarded empirical acceptance rule that compares refinement against repair.

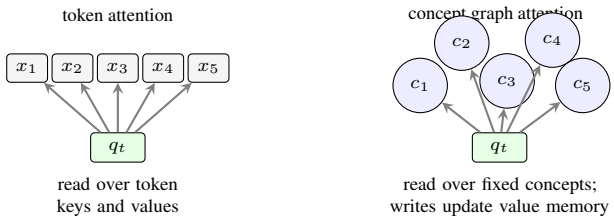

Figure 1: CGA changes the memory object of attention. Standard attention reads from token memory. CGA performs a continuous attention read over fixed concept identities whose value memories are updated causally.

- **Architecture.** We introduce fixed-bank CGA as a scalable sequence-modeling specialization of DGM: attention reads over concept identities, while causal writes update bounded concept value memory at inference time.
- **Evaluation.** We implement CGA in the PoST/Zoology framework and evaluate it on state-equalized multi-query associative recall and matched language-model pretraining with zero-shot downstream evaluation.

The claim boundary is important. CGA is not a new tokenizer, loss, or inference-time optimizer. It is a method for changing the memory basis of attention from tokens to runtime concepts. The fixed-bank version in the main experiments deliberately removes online slot allocation so the architectural question is isolated. The appendix restores the broader DGM theory: minimal joins, finite-sample refinement scores, graph data-structure costs, local repair, projection memory, and formal proofs.

2 Online Prediction under Information Partitions

Let Ω be a finite case space, $\omega \sim P$, $Y : \Omega \to \mathcal{Y}$ a consequence variable, A an action space, and $L : \mathcal{Y} \times \mathsf{A} \to \mathbb{R}$ a loss with finite expectations. An information structure is a partition $\Pi = \{B_1, \ldots, B_m\}$. A predictor is Π-measurable when it is constant inside each atom:

$$\mathsf{Pred}(\Pi) = \{f : \Omega \to \mathsf{A} : \omega, \omega' \in B \in \Pi \Rightarrow f(\omega) = f(\omega')\}.$$

The Bayes risk under Π is

$$R_L(\Pi) = \inf_{f \in \mathsf{Pred}(\Pi)} \mathbb{E}[L(Y, f(\omega))] = \sum_{B \in \Pi} P(B) \inf_{a \in \mathsf{A}} \mathbb{E}[L(Y, a) \mid \omega \in B].$$

A repair update changes the payload $a(B)$ assigned to existing atoms; a refinement changes the equivalence relation itself. If a runtime representation is an information map $I : \Omega \to \mathcal{Z}$, then it induces $\Pi(I) = \{I^{-1}(z) : I^{-1}(z) \neq \varnothing\}$, and every predictor of the form $a(I(\omega))$ is $\Pi(I)$-measurable. This is the bridge to neural or memory implementations: the theorem constrains the quotient induced by the runtime state, not the syntax of the optimizer used to update that state. All statements extend to sub-sigma-algebras; the full measure-theoretic version and base-information convention are in Appendix D.

A binary distinction is a prediction-time observable map $U : \Omega \to \{0, 1\}$. We assume admissible distinctions lie in a pre-outcome observable class $\mathcal{U}$, so U_t does not depend on Y_t after observation. The join refinement is

$$\Pi \vee U = \{B \cap U^{-1}(0), B \cap U^{-1}(1) : B \in \Pi\} \setminus \{\varnothing\}.$$

In an online process, $\omega_t \sim P$, the learner predicts with $\hat{f}_t \in \mathsf{Pred}(\Pi_t)$, observes Y_t, and either repairs with $\Pi_{t+1} = \Pi_t$ or refines with $\Pi_{t+1} = \Pi_t \vee U_t$. The optional cost model charges $\lambda c(U_t)$ when a refinement is accepted; repair has no structural cost but may change local memories, sufficient statistics, or downstream parameters.

For any $f_t \in \mathsf{Pred}(\Pi_t)$ and any comparator $\Pi^\star$ refining Π_t,

$$R_L(f_t) - R_L(\Pi^\star) = \underbrace{R_L(f_t) - R_L(\Pi_t)}_{\Delta_{\text{repair}}} + \underbrace{R_L(\Pi_t) - R_L(\Pi^\star)}_{\Delta_{\text{refine}}}.$$

Both terms are nonnegative. The first is residual risk at fixed information; the second is the population value of changing information. Thus the repair/refinement decision is not a choice between two optimizers, but a choice between two admissible prediction classes. For example, if two old atoms merely have poorly estimated Bernoulli probabilities, then Δ_{repair} can dominate and more samples or a better local statistic are enough. If a single old atom mixes two subpopulations with different conditional labels, then Δ_{refine} persists even after the best atomwise statistic has converged. The theorem below formalizes the latter case sequentially.

Theorem 2.1 (Repair-only linear regret)**.** *Let Π be fixed and let $\Pi^\star$ refine Π. Consider any adaptive learner such that, for every t, $\hat{f}_t \in \mathsf{Pred}(\Pi)$, possibly chosen using all previous observations. If $\omega_t \overset{\mathrm{iid}}{\sim} P$, then*

$$\mathbb{E}\left[\sum_{t=1}^{T} L(Y_t, \hat{f}_t(\omega_t))\right] \geq T R_L(\Pi).$$

If Bayes actions exist under $\Pi^\star$, then regret against the refined oracle satisfies

$$\mathbb{E}[\mathrm{Reg}_T(\Pi^\star)] \geq T\left(R_L(\Pi) - R_L(\Pi^\star)\right).$$

If the infimum under $\Pi^\star$ is not attained, then for every $\epsilon > 0$ the same statement holds against an ϵ-optimal refined oracle with right side $T(R_L(\Pi) - R_L(\Pi^\star) - \epsilon)$. If refinement cost is charged once as $\lambda C(\Pi^\star)$, subtract $\lambda C(\Pi^\star)$ from the bound.

Proof. Condition on the history $\mathcal{H}_{t-1}$ before round t. Since $\hat{f}_t \in \mathsf{Pred}(\Pi)$,

$$\mathbb{E}[L(Y_t, \hat{f}_t(\omega_t)) \mid \mathcal{H}_{t-1}] \geq R_L(\Pi).$$

Taking expectation and summing over t gives the first inequality. The refined oracle achieves $R_L(\Pi^\star)$ per round when Bayes actions exist; otherwise choose an ϵ-optimal atomwise predictor under $\Pi^\star$. Subtracting its cumulative risk gives the regret statement, and a one-time refinement charge subtracts from the comparator advantage. □

The theorem is simple but useful: once the information quotient is fixed, no optimizer can exploit distinctions outside it. It applies to caches, local sufficient statistics, adapters, or gradient steps only when they preserve the relevant information map. Backpropagation through a representation can change the quotient and is therefore outside the repair-only lower bound. The refined oracle is also intentionally generous: it receives $\Pi^\star$ for free except for the optional one-time cost term. The result therefore should not be read as a claim that a particular refinement algorithm will find $\Pi^\star$ quickly. It says that if the learner refuses to change the quotient at all, the gap is paid every round.

3 Which Distinctions Matter?

Adding a distinction has a canonical minimal form. For $D = U^{-1}(1)$,

$$\Pi \vee U = \{B \cap D, B \cap D^c : B \in \Pi\} \setminus \{\varnothing\}.$$

Proposition 3.1 (Minimal join)**.** *$\Pi \vee U$ is the unique coarsest refinement of Π that makes U measurable.*

The operation splits exactly the old atoms crossed by the new distinction and leaves all other atoms unchanged. To see minimality, let $\mathcal{Q}$ be any refinement of Π that makes $D = U^{-1}(1)$ measurable. Every atom $Q \in \mathcal{Q}$ lies inside some old atom B and, because D is $\mathcal{Q}$-measurable, lies either in D or in D^c. Hence Q lies inside either $B \cap D$ or $B \cap D^c$, so $\mathcal{Q}$ refines $\Pi \vee U$. This matters operationally: refinement should not be equated with wholesale model growth. The canonical information update adds the least observability required by the proposed bit. Its value is task-dependent:

$$V_L(U \mid \Pi) = R_L(\Pi) - R_L(\Pi \vee U).$$

Risk monotonicity gives $V_L(U \mid \Pi) \geq 0$ at the population optimum, because every old predictor remains feasible after refinement. Finite implementations can still overfit, which is why Section 4 uses held-out scoring and penalties before committing a split.

Theorem 3.2 (Loss-relevant refinement)**.** *Let Π' refine Π. Assume Bayes actions exist and the Π'-Bayes action is unique on positive-mass atoms. Then*

$$R_L(\Pi) = R_L(\Pi')$$

if and only if the Π'-Bayes action has a Π-measurable version. Equivalently, in the finite partition model, zero value means the refined Bayes action is constant across all refined atoms inside each old atom $B \in \Pi$.

Proof sketch. If the refined Bayes action is Π-measurable, it is feasible under Π, so $R_L(\Pi) \leq R_L(\Pi')$; monotonicity gives equality. If risks are equal, any Π-Bayes action is also Π'-optimal. Uniqueness under Π' forces agreement with the refined Bayes action on positive-mass atoms. □

For finite Y under log loss, $R_{\log}(\Pi) = H(Y \mid \Pi)$. The Bayes action in atom B is the empirical or population conditional law $P(Y \mid B)$, and the risk is the conditional entropy averaged over atoms. For a binary distinction U,

$$V_{\log}(U \mid \Pi) = R_{\log}(\Pi) - R_{\log}(\Pi \vee U) = I(Y; U \mid \Pi).$$

Equivalently,

$$V_{\log}(U \mid \Pi) = \sum_{B \in \Pi} P(B) \sum_{u \in \{0,1\}} P(U = u \mid B) D_{\mathrm{KL}}(P(Y \mid B, U = u) \,\|\, P(Y \mid B)).$$

Thus all candidate bits may have the same marginal frequency while only consequence-relevant bits have positive conditional mutual information. This identity is the reason BalancedMask-HQS is a useful diagnostic: the stream makes all query bits equally frequent, so frequency or geometric novelty cannot identify the correct bit, while $I(Y; U \mid \Pi)$ can.

For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss,

$$R_2(\Pi) - R_2(\Pi \vee U) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \Pi, U] - \mathbb{E}[Y \mid \Pi]\|_2^2\right].$$

We use standard value-of-information identities here, specialized to the refine-or-repair online setting [Cover and Thomas, 2006, Gneiting and Raftery, 2007]. The value of a distinction is not geometric novelty; it is consequence value. Consequently, a visually clean cluster split can be worthless if both children have the same Bayes action, while a low-level bit can be valuable if it changes the conditional consequence. This is also why the algorithm compares refinement against repair rather than accepting every statistically nonzero split: a split can have positive value but still be smaller than the gain from simply repairing stale payloads.

4 Guarded Empirical Refinement

Population value must be estimated before a runtime learner commits a new distinction. For an atom B and candidate split $g : B \to \{0, 1\}$, the classification score used in the experiments is empirical log-loss improvement:

$$\widehat{V}_{\log}(B, g) = H(\widehat{p}_B) - \sum_{b \in \{0,1\}} \frac{|B_b|}{|B|} H(\widehat{p}_{B_b}), \qquad B_b = \{i \in B : g(x_i) = b\}.$$

The analogous squared-loss score is $\mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1)$.

For a state S and held-out scoring buffer $\mathcal{B}_t^{\mathrm{score}}$ of size m_t, define

$$\widehat{R}(S; \mathcal{B}_t^{\mathrm{score}}) = \frac{1}{m_t} \sum_{(\omega_i, Y_i) \in \mathcal{B}_t^{\mathrm{score}}} L(Y_i, f_S(\omega_i)), \qquad \widehat{G}_t(S') = \widehat{R}(S_t; \mathcal{B}_t^{\mathrm{score}}) - \widehat{R}(S'; \mathcal{B}_t^{\mathrm{score}}).$$

Let $\mathcal{C}_t^{\mathrm{rep}}$ be repair candidates and $\mathcal{C}_t^{\mathrm{ref}}$ refinement candidates. The best empirical repair gain is $\max_{S' \in \mathcal{C}_t^{\mathrm{rep}}} \widehat{G}_t(S')$. The best penalized refinement gain is $\max_{S' \in \mathcal{C}_t^{\mathrm{ref}}} \{\widehat{G}_t(S') - \lambda c(S')\}$. Candidate proposal may use a disjoint proposal buffer, but the scoring buffer must be held out from proposal and previous state-selection decisions for the concentration statement below. This sample split is the statistical version of the paper's conceptual split: proposal signals can be broad and heuristic, but acceptance is a consequence-scored comparison between repair and refinement. The identity state is included among repair candidates, so the best empirical repair gain is nonnegative and a harmful local update can be declined.

Theorem 4.1 (Guarded empirical refinement)**.** *Assume losses lie in* $[0, 1]$*. At time* t*, let* $\mathcal{C}_t = \mathcal{C}_t^{\mathrm{rep}} \cup \mathcal{C}_t^{\mathrm{ref}}$ *be predictable from the past and proposal buffer, and let* $\mathcal{B}_t^{\mathrm{score}}$ *contain* m_t *fresh held-out samples independent of candidate selection. Define*

$$\epsilon_t = \sqrt{\frac{\log\big(4t^2(|\mathcal{C}_t| + 1)/\delta\big)}{2m_t}}.$$

Then with probability at least $1 - \delta$*, uniformly over all* t *and* $S' \in \mathcal{C}_t$*,*

$$\left|\widehat{G}_t(S') - G_t(S')\right| \le 2\epsilon_t.$$

Consequently, if a refinement S_f *is accepted because*

$$\widehat{G}_t(S_f) - \lambda c(S_f) > \widehat{G}_t(S_r) + \tau + 4\epsilon_t,$$

where S_r *is the best empirical repair candidate, then*

$$G_t(S_f) - \lambda c(S_f) > G_t(S_r) + \tau.$$

Proof sketch. Condition on the past filtration; the old state and candidate states are fixed. Hoeffding controls $|\widehat{R}(S) - R(S)|$ for the old state and all candidates, and a union bound over $|\mathcal{C}_t| + 1$ states and over t with the t^{-2} schedule gives the displayed high-probability event. A gain is a difference of two risks, hence its deviation is at most $2\epsilon_t$. Comparing a refinement gain to a repair gain subtracts two gain deviations, which is why the guard uses $4\epsilon_t$. □

The theorem is a finite-sample guard, not a global training theorem. If the same validation buffer is reused adaptively, one needs reusable-holdout or adaptive-validation control; otherwise the rule should be read as an empirical heuristic. The margin also clarifies the role of the threshold τ. The $4\epsilon_t$ term accounts for sampling uncertainty in the empirical comparison; τ is a modeling margin that can absorb unmodeled costs, numerical instability, or a desired minimum effect size. In CGA, $c(S')$ charges concept and edge growth, while repair candidates keep the accepted distinction set fixed. The result assumes a finite candidate class at the scoring time. Appendix H gives broader greedy-recovery and oracle-inequality statements, including a local-split bound whose atom count grows as $m_0 + B$ rather than $m_0 2^B$.

5 Concept Graph Attention

CGA is the fixed-capacity sequence layer induced by the preceding refine-or-repair view. The general DGM picture allows the information structure to grow by accepting new distinctions; the fixed-bank CGA used here instead allocates a bounded set of concept identities in advance and studies whether forward-time writes into those identities can replace token-indexed attention memory. In that sense, a concept slot is an atom of runtime information with a trainable addressing rule and a mutable payload. A token can repair the payload by writing a new value; training can learn better read/write projections; and the concept budget C controls how many runtime distinctions the layer can keep active.

Operationally, CGA is an attention layer whose memory is a fixed bank of concept identities rather than a table of token keys and values. The "graph" is the runtime bipartite interaction between tokens and concept nodes, with soft read and write weights computed by attention. Each layer has C concept slots and H heads. For head h, slot j has a fixed unit address

$$a_{hj} \in \mathbb{R}^{d_a}, \qquad \|a_{hj}\|_2 = 1.$$

The addresses are initialized once and remain fixed; they are concept identities, not online sequence state. The online state for a sequence is the value memory

$$M_b \in \mathbb{R}^{H \times C \times d_v^c},$$

where b indexes a causal processing block and d_v^c is the per-head concept value width. Thus the persistent runtime state per layer is

$$S_{\mathrm{CGA}} = CHd_v^c.$$

This is the main state-size difference from token attention: attention stores a new key-value entry for every token, while CGA stores a bounded value table over fixed concept nodes.

Projection and normalization. Given hidden states $x_t \in \mathbb{R}^{d_{\text{model}}}$, CGA forms normalized read queries and write keys, concept values, write gates, and retention rates:

$$q_{th} = \text{normalize}(Q_h x_t), \qquad k_{th} = \text{normalize}(K_h x_t),$$
$$v_{th} = V_h x_t, \qquad g_t = \sigma(w_g^\top x_t + b_g).$$

The query and key maps learn how tokens address the fixed concept identities. The value map learns what payload should be stored in concept memory, and the write gate learns how much a token should modify memory.

Dense concept read. CGA reads by applying the standard attention softmax over concept slots rather than over prefix tokens. Let $\gamma > 0$ be a learned read scale. For token t in block b,

$$\alpha_{thj} = \frac{\exp(\gamma q_{th}^\top a_{hj})}{\sum_{r=1}^{C} \exp(\gamma q_{th}^\top a_{hr})}, \qquad z_{th} = \sum_{j=1}^{C} \alpha_{thj} M_{b,hj}.$$

The multi-head output is

$$y_t = O[z_{t1}; \ldots; z_{tH}].$$

This is ordinary differentiable attention except that the keys are fixed concept identities and the values are runtime concept memories.

Continuous concept write. CGA uses a read-before-write protocol. The tokens in a block first read the memory state M_b; after those outputs are emitted, the block writes its values into memory to produce M_{b+1}. For a write delay $\delta \geq 0$, define

$$\bar{k}_{th} = k_{t-\delta,h},$$

with invalid delayed positions masked out. MQAR uses $\delta = 1$ so that a key token can bind to the following value token; general language modeling can use $\delta = 0$. The write distribution is another concept softmax:

$$\beta_{thj} = \frac{\exp(\gamma \bar{k}_{th}^\top a_{hj})}{\sum_{r=1}^{C} \exp(\gamma \bar{k}_{th}^\top a_{hr})}.$$

For block $b = [s, e)$, CGA updates each concept value by

$$M_{b+1,hj} = \rho_{bh} M_{b,hj} + \sum_{t=s}^{e-1} \mathbf{1}_{t \geq \delta}\, g_t\, \beta_{thj}\, v_{th}.$$

The write is continuous: every valid token can write to every concept with a different weight, and the soft weights make the write path differentiable during training.

Trainable retention. The factor $\rho_{bh} \in (0, 1]$ controls how much old concept memory survives when a new block is written. CGA computes tokenwise positive decay rates λ_{th} and accumulates them over the block:

$$\rho_{bh} = \exp\left(-\sum_{t=s}^{e-1} \lambda_{th}\right).$$

Our default parameterization uses a PoST-style ordered spectrum for the base decay rates:

$$A_h = A_0 + \sum_{i<h} \text{softplus}(\Delta_i),$$

combined with a learned input-dependent step size and optional position-adaptive scaling. This gives CGA the same kind of trainable forgetting mechanism used by modern recurrent mixers, but the recurrence acts on concept value memory rather than on token-indexed state.

Training and inference. During training, CGA is optimized end-to-end with the task loss. Gradients flow through the read weights α, the write weights β, the written values v_t, the write gate g_t, and the retention factors ρ_{bh}. The concept addresses a_{hj} are fixed identities, so the model learns to use them by changing the query, key, value, gate, retention, and output projections. During inference, the learned parameters are fixed. The concept memory M_b is still updated by the same forward rule, so future tokens condition on information written from the observed prefix. This is the sense in which CGA performs test-time learning: it learns the contents of a bounded memory during the forward pass, without running backpropagation at inference time.

Algorithm 1 CONCEPTGRAPHATTENTIONFORWARD

Input: hidden states $x_{1:T}$, fixed concept addresses $a_{1:C}$, block size B.
Initialize value memory $M_0 \leftarrow 0$.
for causal blocks $b = [s, e)$ **do**
  Project block queries q, write keys k, values v, gates g, and retention rates λ.
  Read M_b with dense concept attention over fixed addresses.
  Emit block outputs through the output projection.
  Form delayed write keys $\tilde{k}$ and write weights β.
  Compute block retention $\rho_b = \exp(-\sum_{t=s}^{e-1} \lambda_t)$.
  Update $M_{b+1} \leftarrow \rho_b M_b + \sum_{t=s}^{e-1} g_t \, \beta_t \, v_t$.
end for
Return projected concept readouts.

Relation to ordinary attention. Standard causal attention computes

$$\mathrm{Attn}(X)_t = \sum_{s<t} \mathrm{softmax}_s(q_t^\top K_s) V_s,$$

where the memory basis is the set of prefix token positions. CGA computes

$$\mathrm{CGA}(X)_t = \sum_{j=1}^{C} \mathrm{softmax}_j(q_t^\top a_j) M_{tj},$$

where the memory basis is a fixed set of concept identities and only the value contents M_{tj} evolve over the prefix. Thus CGA is not a different loss or optimizer around attention. It is an attention replacement that moves the read/write memory from tokens to concepts.

6 Experiments

We evaluate CGA as a causal sequence mixer: a layer with the external contract of attention, but whose memory is a bounded concept bank rather than all previous tokens. The empirical program has two tracks. First, multi-query associative recall (MQAR) isolates the key mechanism: writing key-value information into runtime memory and retrieving it later under matched state budgets. Second, matched language-model pretraining tests whether the same concept read/write layer remains useful in ordinary next-token prediction. The LM protocol follows the PoST paper's evaluation style so that CGA can be compared directly against Mamba-2, RWKV-7, and Gated DeltaNet runs trained on the same data and evaluated with the same harness.

6.1 Experiment 1: Multi-Query Associative Recall

MQAR inputs contain key-value pairs followed by queries; the model must output the corresponding value [Arora et al., 2024]. This is the cleanest test for CGA because the task separates two questions: whether the model can store associations in a runtime memory, and whether later queries can retrieve them. For token attention, the memory is the raw prefix. For CGA, the memory is a fixed concept bank whose value contents are written from the prefix.

We run CGA inside the existing PoST/Zoology MQAR framework. The comparison is state-equalized against existing efficient sequence mixers. We use the same grid as the PoST/Mamba benchmark runs:

$$(d_{\mathrm{model}}, H) \in \{(512, 4), (512, 8), (256, 4)\}, \qquad d_{\mathrm{state}} = d_{\mathrm{model}}/(2H).$$

These correspond to target runtime-state budgets $d_{\mathrm{model}}^2/H \in \{64\mathrm{K}, 32\mathrm{K}, 16\mathrm{K}\}$. For MQAR, a concept slot is intended to hold an association, so the target concept capacity is tied to the largest training key-value count rather than to the sequence length. The curriculum reaches 128 key-value pairs, so the main grid sets $C = 128$ for every CGA configuration.

For each grid point we choose a per-head concept value bottleneck d_v^c so that the persistent CGA state

$$S_{\mathrm{CGA}} = CHd_v^c$$

does not exceed d_{model}^2/H. This is the runtime value state stored by fixed-bank CGA; concept identities are fixed layer parameters and are not counted as online sequence state. The value bottleneck is rounded down to an eight-dimensional alignment. This gives $(C, d_v^c) = (128, 128)$ for $(512, 4)$, $(128, 32)$ for $(512, 8)$, and $(128, 32)$ for $(256, 4)$, matching the 64K, 32K, and 16K state budgets. Each grid point is run with the same three-point learning-rate sweep

$$\{10^{-3}, 10^{-2.5}, 10^{-2}\}.$$

The implementation uses dense concept reads, untied read/write address projections, a learned read logit scale initialized at the standard dot-product attention scale, a learned write gate with sparsity coefficient 10^{-4}, fixed concept identities, and continuous value writes. The main run uses the PoST parameterization for the retention spectrum, with $\mathrm{dt} \in [10^{-6}, 3 \cdot 10^{-5}]$, train length 512, and position-adaptive scaling enabled. Training uses sequence length 512 with the same curriculum over key-value count as the PoST baselines. Evaluation reports the existing persist and capacity slices at lengths $512, 1024, 2048, 4096$.

Baselines. The comparison class is the standard MQAR suite already present in the lab framework:

- **Transformer**: token-level causal self-attention.
- **Mamba/RWKV/GDN/GLA/RetNet**: recurrent or linear-state efficient mixers.
- **TTT-style layers**: expressive state updated by test-time learning style mechanisms.
- **CGA**: attention over a fixed concept memory.

What the experiment tests. The fair-state run asks whether replacing token memory by concept memory is competitive at the same state budget. The important diagnostic slices are:

$$(\text{test length},\ \text{key-value count},\ \text{persist/capacity}).$$

A positive result is not merely low training loss. CGA should maintain recall when the query distance grows, expose a compact runtime state, and report latency and concept count so that improvements are not hidden inside unbounded memory. We additionally log the mean write gate, valid-token write gate, write mass per token, read scale, mean retention, and mean decay rate. These diagnostics are part of the main run because the central failure mode for CGA on MQAR is not only optimization loss, but whether useful key-value writes are routed into the fixed concept bank and whether the retention time constants are compatible with the sequence length.

Table 1: **MQAR capacity accuracy (%)** at each test length T with $K{=}T/4$ key-value pairs. All models are trained at $T{=}512$; longer lengths are out-of-distribution.

	state= $64K$					state= $32K$					state= $16K$				
Model	512	1K	2K	4K	**Avg**	512	1K	2K	4K	**Avg**	512	1K	2K	4K	**Avg**
Mamba-2	100.0	96.8	62.2	18.9	69.5	99.2	85.2	41.3	11.6	59.4	99.3	80.6	31.2	5.7	54.2
RWKV-7	100.0	100.0	96.1	39.2	83.8	100.0	100.0	80.1	9.5	72.4	100.0	95.2	46.0	10.8	63.0
Gated DeltaNet	100.0	100.0	92.0	42.4	83.6	100.0	96.4	56.7	15.9	67.2	99.8	82.7	31.7	7.4	55.4
GLA	100.0	97.8	67.2	20.8	71.5	100.0	97.7	50.3	7.6	63.9	99.8	88.5	38.7	7.8	58.7
RetNet	99.9	47.1	2.3	0.0	37.3	99.9	63.2	6.0	0.3	42.3	96.8	16.8	0.7	0.0	28.6
CGA (ours)	**100.0**	**100.0**	**99.9**	**96.1**	**99.0**	**100.0**	**100.0**	**97.7**	**72.8**	**92.6**	**100.0**	**98.9**	**86.4**	**45.8**	**82.8**

The CGA row reports the best single epoch at each state size, selected by the average of the four displayed MQAR capacity accuracies over the learning-rate sweep. The selected runs use learning rates $10^{-2.5}$, 10^{-2}, and 10^{-3} for the 64K, 32K, and 16K state budgets, respectively.

The implementation artifact is `../PoST_dev/zoology/zoology/experiments/cga_mqar.py`. The registered mixer is `zoology.mixers.cga.ConceptGraphAttention`.

6.2 Experiment 2: Language-Model Pretraining and Evaluation

The language-model experiment is the scaling test. It asks whether fixed concept memory can replace token attention inside a standard decoder and remain competitive with modern efficient

recurrent mixers. We follow the PoST language-model protocol: pretrain base models on FineWeb-Edu [Lozhkov et al., 2024] at context length $T_{\text{train}} = 2048$, then evaluate zero-shot with the Language Model Evaluation Harness [Gao et al., 2024]. The primary scale is 180M parameters; a 440M run is the direct scaling check on $8\times$H200 hardware.

CGA-LM model. CGA-LM uses the same decoder scaffold as the baseline language models: token embeddings, residual blocks, normalization, and tied LM head are standard. Only the causal mixer is changed: the attention or recurrent mixing sublayer is replaced by CGA. The main state-matched runs use the mixer-only residual scaffold of the Mamba-2 baseline; a Transformer-style MLP is supported but is not part of the primary comparison. For a state-matched comparison to the PoST Mamba-2 configurations, we set

$$C = 128, \qquad d_v^c = 64, \qquad S_{\text{CGA}} = CHd_v^c.$$

At 180M, using $H = 24$ gives $S_{\text{CGA}} = 196{,}608$ values per layer, matching the Mamba-2 state budget 1536×128. At 440M, using $H = 32$ gives $S_{\text{CGA}} = 262{,}144$, matching 2048×128. Concept identities are fixed layer parameters and are not counted as online runtime state. The main CGA-LM run uses fixed concept identities, dense concept reads, continuous value writes, a learned write gate, and PoST-style retention parameterization. The implementation artifact is `../PoST_dev/models/cga_lm.py`; training is launched through the same `../PoST_dev/trainer.py` entry point as the PoST baselines using `-arch cga`.

Metrics. We report validation loss and perplexity during pretraining, then downstream zero-shot results on the same task set as PoST: LAMBADA, HellaSwag, PIQA, ARC-Easy, ARC-Challenge, WinoGrande, and OpenBookQA. For systems comparison, each run should also report training tokens per second, prefill latency, decode latency, parameter count, online state size, concept count, mean write gate, read scale, and retention statistics. The speed numbers are not secondary: CGA's claim is an attention-compatible memory replacement, so the correct comparison includes both quality and sequence-processing cost.

Table 2: **Downstream zero-shot language-model evaluations.** Baseline and PoST rows reproduce the PoST LM evaluation table. CGA-LM rows are reserved for fixed-bank CGA runs trained with the matched FineWeb-Edu protocol and evaluated with the same LM Harness tasks.

Model	LAMBADA acc↑	LAMBADA ppl↓	HellaSwag acc_n ↑	PIQA acc↑	ARC-Easy acc↑	ARC-Challenge acc_n ↑	WinoGrande acc↑	OpenBookQA acc_n ↑	Avg
Mamba-2 180M	**21.6**	**145.4**	31.1	62.9	**50.4**	24.7	49.6	**30.6**	38.7
+PoST	21.5	148.2	**31.3**	**63.2**	50.1	**24.9**	**50.6**	30.0	**38.8**
RWKV-7 180M	27.9	**69.6**	32.1	**63.1**	49.7	**25.7**	51.3	29.0	39.8
+PoST	**28.3**	71.9	32.1	62.9	**52.1**	25.3	**51.8**	**32.0**	**40.6**
Gated DeltaNet 180M	23.8	**94.5**	**31.9**	62.7	49.6	24.2	**51.1**	30.6	39.1
+PoST	**25.2**	95.6	31.5	**62.9**	**51.5**	**25.3**	50.7	**31.8**	**39.8**
CGA-LM 180M (ours)	–	–	–	–	–	–	–	–	–
Mamba-2 440M	24.1	77.3	**37.7**	65.3	**57.7**	**27.2**	50.4	**32.8**	42.2
+PoST	**28.0**	**62.6**	37.5	65.3	56.6	26.2	**51.4**	32.6	**42.5**
CGA-LM 440M (ours)	–	–	–	–	–	–	–	–	–

The main reported comparison is the best CGA-LM checkpoint at each scale against the best baseline row at the same scale. We do not need a broad ablation suite for the first large run. The decisive experiment is whether one clean CGA-LM configuration can match or exceed these baseline averages while using a bounded concept memory and competitive latency.

7 Related Work

Efficient sequence mixers and expressive state. Transformers use content-addressed attention over previous tokens [Vaswani et al., 2017]. Mamba, RWKV, GDN, GLA, RetNet, and related models replace attention with efficient recurrent or linear state updates [Gu and Dao, 2023, Peng et al., 2023, Yang et al., 2025, 2024, Sun et al., 2023]. TTT-style models instead make the hidden state expressive by running a learned update rule at test time [Sun et al., 2025]. Our experiments place CGA in this comparison class: CGA is an attention-like causal mixer whose persistent state is a bounded concept memory rather than a token table.

Table 3: **Language-model systems metrics.** Latency is measured from trained checkpoints in BF16 on H200 with the same synthetic-token benchmark for every architecture. Prefill reports tokens/second for a full forward pass; decode reports conservative iterative next-token latency using the public model interface. The CGA rows should be filled from `scripts/benchmark_lm_latency.py`.

Model	Scale	Params	State/layer	Prefill tok/s	Decode ms/token	Peak GB
Mamba-2 + PoST	180M	–	196K	–	–	–
RWKV-7 + PoST	180M	–	–	–	–	–
Gated DeltaNet + PoST	180M	–	–	–	–	–
CGA-LM (ours)	180M	–	196K	–	–	–
Mamba-2 + PoST	440M	–	262K	–	–	–
CGA-LM (ours)	440M	–	262K	–	–	–

Test-time learning layers. TTT layers frame sequence modeling as test-time learning: the hidden state is a small model, and the state update is a self-supervised training step on the current sequence [Sun et al., 2020, 2025]. CGA adopts the same high-level goal, namely to let inference-time computation learn from the observed prefix, but changes the learning object. The hidden state in CGA is not a linear model or MLP whose weights are updated; it is a concept-indexed value memory. The update is therefore forward-only at inference: read concepts, decay old values, and add gated continuous writes. This makes CGA complementary to TTT. TTT explores expressive hidden states as trainable inner models; CGA explores expressive hidden states as writable concept memories with attention-style readout.

Attention variants. Sparse attention, sliding-window attention, grouped-query attention, and compressed-memory attention mainly alter which token keys and values are read or how the token table is stored. CGA is orthogonal: it changes the basis being attended to. The read still has the same softmax form, so optimized attention kernels and attention variants can in principle be adapted to concept slots instead of token positions.

Associative and external memory. Cache language models, kNN-LMs, retrieval-augmented systems, fast weights, online ridge or RLS memories, Memory Networks, Neural Turing Machines, and Differentiable Neural Computers all use forms of stateful retrieval or local memory update [Grave et al., 2017, Khandelwal et al., 2020, Lewis et al., 2020, Ba et al., 2016, Schlag et al., 2021, Crammer et al., 2006, Weston et al., 2015, Graves et al., 2014, 2016]. CGA shares the associative-memory goal but keeps the decoder-block interface of attention: each token reads from concept slots and writes back through a gated causal update.

Writable capacity. Mixture-of-experts, Switch Transformers, and Product-Key Memory route tokens or queries to trained capacity [Shazeer et al., 2017, Fedus et al., 2022, Lample et al., 2019]. CGA also weights writes across concept slots, but the writable capacity is runtime value state rather than a bank of trained expert parameters. The fixed concept count C makes this capacity explicit and state-equalizable against recurrent mixers.

8 Limitations and Conclusion

- CGA's dense read is stable at small concept budgets, but large concept banks will need fused kernels and eventually sparse concept retrieval.
- The current retention decay is trainable and PoST-parameterized by default, but its best time-scale allocation for large language modeling remains empirical.
- Fixed-bank CGA removes online slot-allocation complexity, but it makes the concept count C a real capacity hyperparameter.
- CGA's test-time learning is prefix-local in the current formulation: the concept value memory is reset per sequence unless a separate persistent-memory protocol is added.
- The 180M–440M language-model runs are the intended SOTA-readiness test, but they still require completed training and latency reporting before the paper can claim a large-scale win.

The empirical claim is deliberately scoped: CGA is evaluated as an attention-like causal sequence mixer on MQAR and matched language-model pretraining. MQAR should show whether the concept bank can write and retrieve key-value information under matched state budgets. CGA-LM should show whether the same mechanism can replace token attention in a trainable decoder at 180M–440M scale. If CGA loses to optimized recurrent mixers, that is a real systems or modeling gap rather than a wording problem. The method still needs stronger fused kernels, retention-time-scale studies, and distributed concept-bank implementations before it can be treated as a fully optimized large-model systems result.

The main point is the memory-basis replacement. Attention reads from token-indexed memory. CGA reads from fixed concept identities whose value memories are updated by a causal forward-time rule during inference. This gives a clean attention-compatible path for testing whether concept-level test-time learning can compete with token memory, recurrent state, and TTT-style hidden-model updates under matched state budgets.

References

Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *International Conference on Learning Representations*, 2024.

Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

James O. Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer, 2 edition, 1985.

Albert Bifet and Ricard Gavaldà. Adaptive learning from evolving data streams. In *Advances in Intelligent Data Analysis VIII*, volume 5772 of *Lecture Notes in Computer Science*, pages 249–260. Springer, 2009. doi: 10.1007/978-3-642-03915-7_22.

Patrick Billingsley. *Probability and Measure*. Wiley, 3 edition, 1995.

David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24 (2):265–272, 1953. doi: 10.1214/aoms/1177729032.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240, 2022.

Lev M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967. doi: 10.1016/0041-5553(67)90040-7.

Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

Gail A. Carpenter and Stephen Grossberg. ART 2: Self-organization of stable category recognition codes for analog input patterns. *Applied Optics*, 26(23):4919–4930, 1987a. doi: 10.1364/AO.26.004919.

Gail A. Carpenter and Stephen Grossberg. A massively parallel architecture for a self-organizing neural pattern recognition machine. *Computer Vision, Graphics, and Image Processing*, 37(1): 54–115, 1987b. doi: 10.1016/S0734-189X(87)80014-2.

Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006. doi: 10.1017/CBO9780511546921.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2 edition, 2006.

Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.

Luc Devroye, László Györfi, and Gábor Lugosi. *A Probabilistic Theory of Pattern Recognition*. Springer, 1996.

Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 71–80, 2000. doi: 10.1145/347090.347107.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in adaptive data analysis and holdout reuse. In *Advances in Neural Information Processing Systems*, volume 28, pages 2350–2358, 2015a.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015b. doi: 10.1126/science.aaa9375.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997. doi: 10.1006/jcss.1997.1504.

Bernd Fritzke. A growing neural gas network learns topologies. In *Advances in Neural Information Processing Systems*, volume 7, pages 625–632, 1995.

Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, et al. A framework for few-shot language model evaluation. https://github.com/EleutherAI/lm-evaluation-harness, 2024.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

Edouard Grave, Moustapha M. Cisse, and Armand Joulin. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014.

Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626): 471–476, 2016. doi: 10.1038/nature20101.

Stephen Grossberg. Adaptive resonance theory: How a brain learns to consciously attend, learn, and recognize a changing world. *Neural Networks*, 37:1–47, 2013. doi: 10.1016/j.neunet.2012.09.017.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017.

Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830.

Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, pages 604–613, 1998. doi: 10.1145/276698.276876.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. doi: 10.1109/TBDATA.2019.2921572.

Olav Kallenberg. *Foundations of Modern Probability*. Springer, 2 edition, 2002.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165, 2020.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114.

Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5338–5348, 2020.

Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990. doi: 10.1109/5.58325.

Teuvo Kohonen. *Self-Organizing Maps*, volume 30 of *Springer Series in Information Sciences*. Springer, 1995.

Balaji Lakshminarayanan, Daniel M. Roy, and Yee Whye Teh. Mondrian forests: Efficient online random forests. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. FineWeb-Edu: The finest collection of educational content the web has to offer. *Hugging Face Blog*, 2024. https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1.

Chaitanya Manapragada, Geoffrey I. Webb, and Mahsa Salehi. Extremely fast decision tree. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1953–1962, 2018. doi: 10.1145/3219819.3220005.

Thomas M. Martinetz and Klaus J. Schulten. A neural-gas network learns topologies. In *Artificial Neural Networks*, pages 397–402, 1991.

Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Yaofo Chen, Shihua Zheng, Peilin Zhao, and Mingkui Tan. Efficient test-time model adaptation without forgetting. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16888–16905, 2022.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, et al. RWKV: Reinventing RNNs for the transformer era. *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

Roger Penrose. A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, 51(3):406–413, 1955. doi: 10.1017/S0305004100030401.

J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986. doi: 10.1007/BF00116251.

Jack W. Rae, Jonathan J. Hunt, Tim Harley, Ivo Danihelka, Andrew Senior, Greg Wayne, Alex Graves, and Timothy Lillicrap. Scaling memory-augmented neural networks with sparse reads and writes. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2016.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366, 2021.

Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends in Machine Learning*, 4(2):107–194, 2012. doi: 10.1561/2200000018.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Andrey N. Tikhonov and Vasiliy Y. Arsenin. *Solutions of Ill-Posed Problems*. Winston, 1977.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Vladimir G. Vovk. Aggregating strategies. In *Proceedings of the Third Annual Workshop on Computational Learning Theory*, pages 371–383. Morgan Kaufmann, 1990.

Volodya Vovk. Competitive on-line linear regression. In *Advances in Neural Information Processing Systems*, volume 10, 1997.

Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.

Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *International Conference on Machine Learning*, 2024.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations*, 2018.

Marvin Zhang, Sergey Levine, and Chelsea Finn. MEMO: Test time robustness via adaptation and augmentation. In *Advances in Neural Information Processing Systems*, volume 35, pages 38629–38642, 2022.

Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the 20th International Conference on Machine Learning*, pages 928–936, 2003.

Appendix

A Appendix Roadmap

The appendix keeps secondary theoretical and implementation material out of the main method story. It first states limitations and extended related work, then preserves the full formal model, the longer refinement and concept-graph development, implementation extensions, the projection-memory bridge, and formal proofs.

- **Part 1.** Appendix B clarifies what the theory proves and what it leaves to modeling, statistics, and experiments.
- **Part 2.** Appendix C gives the extended related-work discussion.
- **Part 3.** Appendices D–J retain the full model, value identities, and graph-refinement formalism.
- **Part 4.** Appendix K gives CGA-LM as an attention-replacement architecture; Appendix N collects tree special cases, data-structure costs, the optional adapter variant, systems notes, and closed-form local repair.
- **Part 5.** Appendix M gives the Projection Memory and CPM bridge.
- **Part 6.** Appendices O, P, and Q give the formal proofs.

B Limitations

The theory identifies when a distinction is necessary or valuable at the population-information level. It does not solve the search problem of finding the best distinction in a rich hypothesis class. The online algorithm is greedy and local: its refinement rule concerns local concepts and their nearby conflicts, not global optimality over all information structures.

The risk identities assume correctly estimated conditional distributions or conditional means. With finite data, refinement can increase estimation error, so useful population distinctions may be statistically unsafe. Complexity penalties, thresholds, validation, Bayesian priors, or minimum leaf sizes are needed in practical systems.

The finite-sample validation theorem assumes a genuinely held-out scoring buffer: fresh samples, or samples not previously used to choose proposals or update the state. Reusing the same validation buffer after previous accept/reject decisions makes the candidate class adaptive to that buffer and requires reusable-holdout or other adaptive-validation machinery beyond the Hoeffding argument stated here.

The base-information convention is restricted. If the full hidden state is made freely conditionable in the Bayes-risk benchmark, then all CGA edge predicates and anchor-order events are already measurable and the sigma-algebra refinement gap vanishes. The strict partition theory therefore applies to the current runtime information or to restricted hard-route prediction classes; h-dependent soft memories are implementation approximations or restricted-class extensions.

The CGA method specifies one canonical refinement mechanism, based on concepts and local distinction edges, but this is not claimed to be optimal. Rich domains may require learned concept-creation rules, approximate nearest-neighbor indices, nonlinear distinction edges, and better merge, forgetting, or retention-decay policies than the default budget notes in Appendix N. The bridge to Projection Memory and CPM is exact for one-hot finite partition memories. Continuous neural representations need additional modeling choices: when a continuous conflict should create a new concept, how local projection memories should be inherited, how retention time scales should be allocated, and how refined information structures interact with differentiable backbones.

CGA's dense read avoids sparse candidate-recall assumptions at small concept budgets. Large-budget CGA will likely need sparse concept retrieval, at which point candidate recall again becomes an engineering assumption that should be measured in large systems. Edge gates can reject incompatible candidates, but they cannot select a compatible concept that the retrieval layer never exposes.

The large-model layer is an architectural path rather than a completed scaling theorem. It gives a trainable read path, a gated write path, a trainable retention decay, and a default budget policy with

explicit complexity, but it does not prove optimization guarantees for deep nonconvex backbones, nor does it solve distributed systems issues such as load balancing or stale concept shards. Budget maintenance is also outside the formal refinement theorem: merge, eviction, decay, and compression may discard distinctions, so their effect must be evaluated as a resource-management approximation.

The experiments are a focused sequence-modeling benchmark suite, not a full large-model scaling result. They should not be read as evidence that CGA already dominates optimized attention, Mamba, RWKV, GDN, or TTT implementations at production scale. A complete empirical evaluation should include state-equalized MQAR, language modeling at multiple parameter and token budgets, latency and memory curves, and ablations of repair-only, refine-only, edge-free, and edge-gated variants.

C Related Work

The relevant comparison is not whether CGA uses centers, memory, routing, or online updates. All of these ingredients have long histories. The question is whether a runtime learner distinguishes two operations that are often collapsed: repairing state inside the current representation and refining the representation when consequences reveal a missing distinction.

C.1 Methods that repair without refining

Cache language models and kNN-LMs store or retrieve context-token examples at inference time [Grave et al., 2017, Khandelwal et al., 2020]. Memorizing Transformers, RETRO, RAG, and sparse retrieval systems extend this idea with larger or external stores [Wu et al., 2022, Borgeaud et al., 2022, Lewis et al., 2020, Rae et al., 2016]. These methods make past examples available, but the default response to surprise is more storage or retrieval, not the creation of a new runtime concept forced by consequence conflict.

Fast weights, linear attention memories, delta-rule updates, passive-aggressive learning, online ridge/RLS, and Projection Memory repair associative state inside a fixed geometry [Ba et al., 2016, Katharopoulos et al., 2020, Schlag et al., 2021, Crammer et al., 2006, Vovk, 1997, Bregman, 1967, Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970]. Adapter modules and LoRA provide parameter-efficient adaptation state in large pretrained models [Houlsby et al., 2019, Hu et al., 2022]. Conservative Projection Memory (CPM) is especially close to our repair side because it detects zero innovation with nonzero residual as a conflict. CGA uses this diagnosis differently: a zero-innovation conflict is evidence that the current information cell is too coarse, so projection-style repair should stop until the information structure is refined.

Test-time adaptation and test-time training update model state or parameters by optimizing an inference-time objective [Sun et al., 2020, Wang et al., 2021, Niu et al., 2022, Zhang et al., 2022, Sun et al., 2025]. These methods ask what surrogate loss should be optimized at test time. CGA asks a prior structural question: is the error repairable under the current distinctions, or does it reveal a missing distinction?

These methods solve the repair-gap side of runtime learning. They do not, by themselves, provide an explicit consequence-forced refinement operation.

C.2 Methods that refine without local repair

Decision trees, online trees, Hoeffding trees, adaptive trees, extremely fast decision trees, and Mondrian forests refine leaves using information gain, variance reduction, or streaming split tests [Quinlan, 1986, Breiman et al., 1984, Domingos and Hulten, 2000, Bifet and Gavaldà, 2009, Manapragada et al., 2018, Lakshminarayanan et al., 2014]. These are the closest algorithmic relatives of CGA's refinement test. The difference is structural: trees store distinctions in a rooted hierarchy, while CGA stores local pairwise distinction edges between runtime concepts and attaches repairable memories to the resulting nodes.

Adaptive Resonance Theory, including ART 1 for binary patterns and ART 2 for analog input patterns, addresses stable incremental category creation under mismatch and vigilance [Carpenter and Grossberg, 1987b,a, Grossberg, 2013]. Prototype methods, LVQ, SOMs, neural gas, growing neural gas, and Prototypical Networks represent regions or classes by stored centers or exemplars [Kohonen, 1990, 1995, Martinetz and Schulten, 1991, Fritzke, 1995, Snell et al., 2017]. CGA shares

the vocabulary of category creation and prototypes, but its concepts are operational hidden-state concepts whose edges are consequence-forced binary generators. The node is not only a region; it is a place where local memory can be repaired after the distinction is accepted.

Concept Bottleneck Models and Formal Concept Analysis emphasize explicit concept structures [Koh et al., 2020, Ganter and Wille, 1999]. CGA does not claim human-interpretable concepts by default. Its concepts are runtime concepts: hidden-state anchors, centroids, local memories, and pairwise distinctions created from prediction consequences.

These methods solve the refinement half in various forms. They typically do not pair refinement with graph-gated neural local memories that continue to be repaired after the split.

C.3 Methods that route without runtime concept creation

Mixture-of-experts, Switch Transformers, and Product-Key Memory route tokens or queries to sparse trained capacity [Shazeer et al., 2017, Fedus et al., 2022, Lample et al., 2019]. Memory Networks, End-to-End Memory Networks, Neural Turing Machines, and Differentiable Neural Computers learn external read/write controllers [Weston et al., 2015, Sukhbaatar et al., 2015, Graves et al., 2014, 2016]. These systems are important precedents for sparse reads and external memory. CGA differs in write semantics: the routed objects are runtime-created concepts whose edges generate an information structure and whose memories are repaired locally.

Continual learning studies forgetting, replay, regularization, and capacity growth across task sequences [Kirkpatrick et al., 2017, Lopez-Paz and Ranzato, 2017, Rusu et al., 2016, Yoon et al., 2018]. CGA is complementary: it is an instance-level runtime data structure for local concept growth and memory repair, not a complete task-sequence continual learning system.

C.4 CGA's position

CGA combines refinement and repair but keeps them distinct:

$$\Delta_{\text{refine}} \text{ dominates} \Rightarrow \text{refine}, \qquad \Delta_{\text{repair}} \text{ dominates} \Rightarrow \text{repair}.$$

The measure-theoretic formalism follows standard sigma-algebra and conditional-expectation language [Billingsley, 1995, Kallenberg, 2002]. The Bayes-risk order is related to Blackwell comparison and statistical decision theory [Blackwell, 1953, Berger, 1985]. The log-loss and squared-loss value identities are standard information-theoretic and Hilbert-space facts [Cover and Thomas, 2006, Gneiting and Raftery, 2007]. The contribution is to place these facts inside a runtime-learning model in which some errors are repair signals and others are evidence of missing distinctions.

D Mathematical Model: Partitions, Predictors, and Bayes Risk

We first formalize the information available to a runtime learner as a partition of the case space, using the standard link between information, measurability, and admissible decision rules [Blackwell, 1953, Berger, 1985, Billingsley, 1995, Kallenberg, 2002]. The measurable-space notation used later is the direct sigma-algebra generalization, but the computational object is a finite partition.

D.1 Partitions, predictors, and risk

Let Ω be a finite case space, let P be a distribution on Ω, let $Y : \Omega \to \mathcal{Y}$ be the consequence variable, let A be an action space, and let $L : \mathcal{Y} \times \mathsf{A} \to \mathbb{R}$ be a loss with finite expectations for the predictors under discussion.

Definition D.1 (Information partition and partition-measurable predictor)**.** *A finite information partition is a collection of nonempty disjoint atoms*

$$\Pi = \{B_1, \ldots, B_m\}$$

whose union is Ω. It induces the equivalence relation $\omega \sim_\Pi \omega'$ iff ω, ω' lie in the same atom $B \in \Pi$.

A predictor is Π-measurable if it cannot distinguish states inside the same atom:

$$\mathsf{Pred}(\Pi) = \{f : \Omega \to \mathsf{A} : \omega \sim_\Pi \omega' \Rightarrow f(\omega) = f(\omega')\}.$$

Equivalently, a Π-measurable predictor is a map $a : \Pi \to \mathsf{A}$ evaluated on the atom containing ω.

Symbol	Meaning
Ω	finite case space
P	distribution over cases
$Y : \Omega \to \mathcal{Y}$	consequence or label variable
A	action space
$L : \mathcal{Y} \times \mathsf{A} \to \mathbb{R}$	loss
Π	finite information partition of Ω
$\mathsf{Pred}(\Pi)$	predictors measurable with respect to Π
$R_L(f)$	risk of predictor f
$R_L(\Pi)$	Bayes risk achievable under partition Π
$I : \Omega \to \mathcal{Z}$	information map inducing a partition $\Pi(I)$
$U : \Omega \to \{0, 1\}$	binary distinction
$\Pi \vee U$	partition obtained by splitting Π by U
$\mathcal{F}, \mathcal{G}$	sigma-algebras or information structures
$V_L(\mathcal{F}, \mathcal{G})$	value of refining $\mathcal{F}$ to $\mathcal{G}$
d_L^ϵ	loss-dependent distinction complexity
a_j, c_j, M_j	CGA anchor, centroid, and local memory for concept j
$E_t, g_e, \mathcal{P}_t^G$	CGA edge set, edge generator, and graph-generated partition
$\mathcal{B}_t^{\text{score}}$	held-out scoring buffer for empirical refinement tests
$\widehat{\Delta}_{\text{repair}}, \widehat{\Delta}_{\text{refine}}$	empirical repair and refinement gains

Table 4: Notation used throughout the paper. We say Π' is finer than Π if every atom of Π' is contained in an atom of Π.

Definition D.2 (Bayes risk under a partition)**.** *For $f : \Omega \to \mathsf{A}$, write $R_L(f) = \mathbb{E}[L(Y, f)]$. The Bayes risk achievable under partition Π is*

$$R_L(\Pi) = \inf_{f \in \mathsf{Pred}(\Pi)} \mathbb{E}[L(Y, f)] = \sum_{B \in \Pi} P(B) \inf_{a \in \mathsf{A}} \mathbb{E}[L(Y, a) \mid \omega \in B], \tag{1}$$

where atoms with $P(B) = 0$ may be assigned arbitrary actions. Thus $R_L(\Pi)$ is the best possible risk after the representation has committed to the equivalence relation $\sim_\Pi$, i.e., the Bayes risk under constrained information [Berger, 1985].

Definition D.3 (Information map and induced partition)**.** *An information map is a runtime representation $I : \Omega \to \mathcal{Z}$. It induces the partition*

$$\Pi(I) = \{I^{-1}(z) : z \in \mathcal{Z},\ I^{-1}(z) \neq \varnothing\}.$$

Any predictor that factors through I, written $f(\omega) = a(I(\omega))$, is measurable with respect to $\Pi(I)$.

E Online Prediction with Repair and Refinement

The online learner has two update types: repair preserves the current partition, while refinement joins it with a new binary distinction.

Definition E.1 (Binary distinction and partition join)**.** *A binary distinction is a function $U : \Omega \to \{0, 1\}$. Joining Π with U means splitting every atom of Π according to the value of U:*

$$\Pi \vee U = \{B \cap U^{-1}(0), B \cap U^{-1}(1) : B \in \Pi\} \setminus \{\varnothing\}. \tag{2}$$

Definition E.2 (Refinable online prediction problem)**.** *A refinable online prediction problem is a tuple*

$$\mathfrak{P} = (\Omega, \mathcal{Y}, \mathsf{A}, P, L, \mathcal{O}, \mathcal{U}, c),$$

where Ω is finite, $Y : \Omega \to \mathcal{Y}$ is the consequence variable, P is the sampling distribution, $L : \mathcal{Y} \times \mathsf{A} \to \mathbb{R}$ has finite risk for admissible predictors, $\mathcal{O} \subseteq 2^\Omega$ is the pre-outcome observable sigma-algebra, $\mathcal{U} \subseteq \{0, 1\}^\Omega$ is a class of admissible binary distinctions that are $\mathcal{O}$-measurable, and $c : \mathcal{U} \to \mathbb{R}_+$ assigns refinement costs. Bounded losses are required only in finite-sample concentration statements, where they are stated explicitly. The $\mathcal{O}$-measurability condition prevents a prediction-time distinction from depending directly on the consequence label; pure value-of-information examples can be recovered by taking $\mathcal{O} = 2^\Omega$.

The initial runtime partition is assumed to be prediction-time observable:

$$\sigma(\Pi_0) \subseteq \mathcal{O}.$$

Since every accepted distinction $U_t \in \mathcal{U}$ is $\mathcal{O}$-measurable, induction gives $\sigma(\Pi_t) \subseteq \mathcal{O}$ for every reachable partition. Thus observing the current atom is itself a pre-outcome observation, not an additional label-dependent channel.

At round t, the learner maintains $S_t = (\Pi_t, a_t)$ with $a_t : \Pi_t \to \mathsf{A}$. A case $\omega_t \sim P$ is drawn, the learner observes only the atom $B_t \in \Pi_t$ containing ω_t, predicts $a_t(B_t)$, observes $Y_t = Y(\omega_t)$, and then chooses one of two updates:

$$\begin{aligned} \text{Repair}: &\quad \Pi_{t+1} = \Pi_t, \quad a_{t+1} \textit{ may change}, \\ \text{Refine}: &\quad \Pi_{t+1} = \Pi_t \vee U_t \textit{ for some } U_t \in \mathcal{U}. \end{aligned}$$

The cumulative objective is prediction loss plus refinement cost,

$$\mathcal{L}_T = \sum_{t=1}^{T} L(Y_t, a_t(B_t)) + \lambda \sum_{t:\text{Refine}} c(U_t).$$

Definition E.3 (Fixed-information repair update)**.** *Let a parameterized runtime predictor factor through a fixed information map $I : \Omega \to \mathcal{Z}$:*

$$f_\psi(\omega) = a_\psi(I(\omega)).$$

An update of ψ is a fixed-information repair update if I is unchanged. The update rule may be a gradient step, a closed-form sufficient-statistic update, a cache write, or any other optimizer [Rumelhart et al., 1986, Zinkevich, 2003, Shalev-Shwartz, 2012, Grave et al., 2017, Khandelwal et al., 2020, Ba et al., 2016, Schlag et al., 2021]; what makes it repair is the invariance of the induced partition $\Pi(I)$, not the optimizer used.

F The Repair-Only Obstruction

If the partition is fixed, the learner cannot beat the Bayes risk of that partition, no matter how sophisticated the repair update is. This is the fixed-information specialization of the classical decision-theoretic fact that a decision rule cannot exploit information it is not allowed to condition on [Blackwell, 1953, Berger, 1985].

Definition F.1 (Repair gap and refinement gap)**.** *For a concrete predictor $f \in \mathsf{Pred}(\Pi)$ and a candidate refined partition $\Pi^\star$ that refines Π, define*

$$\Delta_{\text{repair}}(f, \Pi) = R_L(f) - R_L(\Pi), \tag{3}$$

$$\Delta_{\text{refine}}(\Pi, \Pi^\star) = R_L(\Pi) - R_L(\Pi^\star). \tag{4}$$

The first gap is the residual error of the current predictor at fixed equivalence relation. The second is the value of changing the equivalence relation.

Theorem F.2 (Runtime error decomposition)**.** *Let $f_t \in \mathsf{Pred}(\Pi_t)$ and let $\Pi^\star$ refine Π_t. Then*

$$R_L(f_t) - R_L(\Pi^\star) = \Delta_{\text{repair}}(f_t, \Pi_t) + \Delta_{\text{refine}}(\Pi_t, \Pi^\star). \tag{5}$$

Both terms are nonnegative.

Proof. Add and subtract $R_L(\Pi_t)$. The repair gap is nonnegative because f_t is feasible for the infimum defining $R_L(\Pi_t)$. The refinement gap is nonnegative because every Π_t-measurable predictor is also measurable with respect to any refinement $\Pi^\star$. □

F.1 Repair-only lower bound

The pointwise contradiction inside a cell becomes a sequential lower bound. If the current partition collapses states that a refined oracle separates, no repair-only learner can avoid paying the corresponding Bayes-risk gap every round.

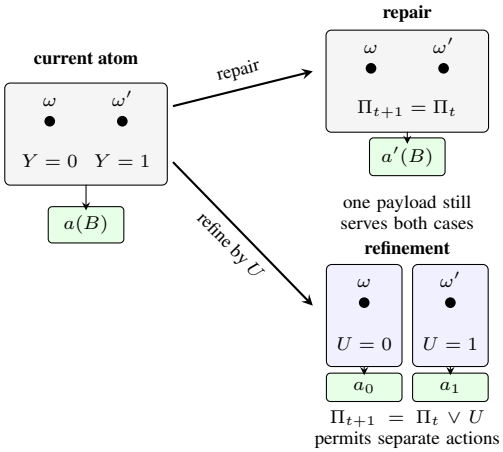

Figure 2: Repair versus refinement. Repair changes the payload assigned to an existing atom but preserves the equivalence relation. Refinement joins the partition with a distinction U, splitting the old atom only where the new bit matters and allowing different local actions.

Theorem F.3 (Repair-only linear regret)**.** *Let Π be a fixed partition and let $\Pi^\star$ be a refinement. Consider any learner whose round-t prediction $\hat{f}_t$ is always in $\mathsf{Pred}(\Pi)$, possibly chosen adaptively from all previous observations. Then, for iid $\omega_t \sim P$,*

$$\mathbb{E}\left[\sum_{t=1}^{T} L(Y_t, \hat{f}_t(\omega_t))\right] \geq T R_L(\Pi).$$

Consequently, if Bayes actions exist under $\Pi^\star$, then relative to an oracle using $\Pi^\star$,

$$\mathbb{E}[\mathrm{Regret}_T] \geq T\left(R_L(\Pi) - R_L(\Pi^\star)\right).$$

If the infimum under $\Pi^\star$ is not attained, the same statement holds in the epsilon-oracle sense: for every $\epsilon > 0$, regret against an ϵ-optimal $\Pi^\star$-measurable oracle is at least $T(R_L(\Pi) - R_L(\Pi^\star) - \epsilon)$. Thus if $R_L(\Pi) > R_L(\Pi^\star)$, every repair-only learner suffers $\Omega(T)$ expected regret against refined or epsilon-refined oracles.

Proof. Condition on the learner's history before round t. The chosen predictor $\hat{f}_t$ is Π-measurable, so by definition of $R_L(\Pi)$,

$$\mathbb{E}[L(Y_t, \hat{f}_t(\omega_t)) \mid \text{history}] \geq R_L(\Pi).$$

Taking expectation and summing over t gives the first inequality. If Bayes actions exist on the atoms of $\Pi^\star$, the $\Pi^\star$ oracle achieves $R_L(\Pi^\star)$ per round by using them, so subtracting its cumulative risk gives the regret bound. Without attained Bayes actions, for every $\epsilon > 0$ choose an ϵ-optimal atomwise predictor under $\Pi^\star$; subtracting its cumulative risk gives the epsilon-oracle bound. □

The displayed comparator gives the refined partition to the oracle for free. If regret includes a one-time refinement cost $C(\Pi^\star)$ for reaching $\Pi^\star$, the same argument gives

$$\mathbb{E}[\mathrm{Regret}_T] \geq T\left(R_L(\Pi) - R_L(\Pi^\star)\right) - \lambda C(\Pi^\star).$$

Corollary F.4 (Fixed-information gradient repair lower bound)**.** *Consider any online optimizer, including backpropagation, whose predictions at every round factor through a fixed information map I:*

$$\hat{f}_t(\omega) = a_{\psi_t}(I(\omega)).$$

Then $\hat{f}_t \in \mathsf{Pred}(\Pi(I))$ for all t, so Theorem F.3 applies with $\Pi = \Pi(I)$. In particular, if a refined partition $\Pi^\star$ has $R_L(\Pi(I)) > R_L(\Pi^\star)$, this fixed-information gradient repair suffers linear regret against the refined oracle. Backpropagation through parameters that change I is outside this corollary; in the present model, such an update changes the information structure rather than merely repairing actions on fixed atoms.

Example F.5 (Two-state separation)**.** *Let $\Omega = \{0, 1\}$, $P(0) = P(1) = 1/2$, and $Y = \omega$. Under the trivial partition $\Pi = \{\{0, 1\}\}$, the best log-loss predictor is $q(Y = 0) = q(Y = 1) = 1/2$, so $R_{\log}(\Pi) = \log 2$. The refined partition $\Pi^\star = \{\{0\}, \{1\}\}$ has $R_{\log}(\Pi^\star) = 0$ in the noiseless case when point-mass predictions are allowed. Therefore any repair-only learner has expected regret $T \log 2$ against the refined oracle. This example is not about neural networks, caches, or trees. It is about equivalence relations: a learner that identifies two states cannot assign them two different consequences.*

Corollary F.6 (Contradiction requires distinction)**.** *Let Π be a partition and let $y : \Omega \to \mathsf{A}$ be a required deterministic output. If ω, ω' lie in the same atom of Π but $y(\omega) \neq y(\omega')$, then no predictor in $\mathsf{Pred}(\Pi)$ satisfies both requirements.*

Proof. Every $f \in \mathsf{Pred}(\Pi)$ is constant on the atom containing ω and ω', so $f(\omega) = f(\omega')$. Satisfying both requirements would imply $y(\omega) = y(\omega')$, a contradiction. □

F.2 Sigma-algebra extension

The finite partition model is the main computational model. The sigma-algebra notation is the measure-theoretic extension used for value identities and language-model interpretations [Billingsley, 1995, Kallenberg, 2002]. In this formulation, L is measurable with finite expectation for the actions considered.

Definition F.7 ($\mathcal{F}$-measurable predictor and Bayes action)**.** *On a probability space $(\Omega, \mathcal{E}, \mathbb{P})$, an information structure is a sub-sigma-algebra $\mathcal{F} \subseteq \mathcal{E}$. For a measurable action space $(\mathsf{A}, \Sigma_{\mathsf{A}})$, a predictor $a : \Omega \to \mathsf{A}$ is $\mathcal{F}$-measurable if*

$$a^{-1}(C) \in \mathcal{F} \qquad \text{for every } C \in \Sigma_{\mathsf{A}}.$$

The admissible predictor class and Bayes risk are

$$\mathsf{Pred}(\mathcal{F}) = \{a : \Omega \to \mathsf{A} : a \text{ is } (\mathcal{F}, \Sigma_{\mathsf{A}})\text{-measurable}\}, \qquad R_L(\mathcal{F}) = \inf_{a \in \mathsf{Pred}(\mathcal{F})} \mathbb{E}[L(Y, a)].$$

When the infimum is attained, any minimizer is an $\mathcal{F}$-Bayes action and is denoted $a^\star_{\mathcal{F}}$. A random variable b has an $\mathcal{F}$-measurable version if there exists an $\mathcal{F}$-measurable random variable $\tilde{b}$ such that $\tilde{b} = b$ almost surely.

For a finite partition Π, Definition F.7 recovers Definition D.1 by taking $\mathcal{F} = \sigma(\Pi)$ and the action space discrete.

Implementations often have fixed pre-existing prediction outputs that are not created by the runtime refinement mechanism, such as a backbone distribution or other fixed pre-outcome features used outside CGA routing. We write this base information as a sigma-algebra $\mathcal{B}$ and measure runtime refinements through the total information $\mathcal{B} \vee \mathcal{F}_t$. The full hidden state used to propose or instantiate distinctions should not automatically be included as freely conditionable base information: if $\mathcal{B}$ contains h, then every edge predicate $g_e(h)$ and anchor-order event is already $\mathcal{B}$-measurable, so the unrestricted sigma-algebra refinement gap becomes trivial. Thus the refinement gap relative to a target $\mathcal{F}^\star$ is

$$R_L(\mathcal{B} \vee \mathcal{F}_t) - R_L(\mathcal{B} \vee \mathcal{F}^\star).$$

Repairs keep $\mathcal{F}_t$ fixed, while refinements replace $\mathcal{F}_t$ by $\mathcal{F}_t \vee \sigma(U_t)$. All monotonicity and value identities below apply after joining both sides with the same fixed $\mathcal{B}$.

Section H uses this notation to characterize exactly when a richer information structure is useful.

G Minimal Refinement

Adding a distinction has a canonical minimal form: split exactly the atoms crossed by that distinction. Once a contradiction shows that the current atom is too coarse, the next question is not how to grow a large model. It is which information must be added. The primitive operation in this paper is minimal refinement: given an information structure and evidence that a new event or variable must be observable, add that observability and nothing else. We state the finite partition theorem first, then the sigma-algebra form. The latter is the standard measure-theoretic join; the former is its finite special case [Billingsley, 1995, Kallenberg, 2002].

G.1 The finite refinement rule

Definition G.1 (Partition-measurable event and finite split)**.** *Let $\mathcal{P}$ be a finite partition of Ω. An event $D \subseteq \Omega$ is $\mathcal{P}$-measurable if it is a union of atoms of $\mathcal{P}$. For any event D, the finite split of $\mathcal{P}$ by D is*

$$\begin{aligned}\mathcal{P}_D = \{B \cap D : B \in \mathcal{P}, B \cap D \neq \varnothing\} \\ \cup \{B \cap D^c : B \in \mathcal{P}, B \cap D^c \neq \varnothing\}.\end{aligned} \tag{6}$$

This keeps exactly the nonempty pieces obtained by cutting each old atom by D.

Theorem G.2 (Minimal finite refinement)**.** *Let $\mathcal{P}$ be a finite partition of Ω and let $D \subseteq \Omega$. The partition $\mathcal{P}_D$ is the unique minimal partition that refines $\mathcal{P}$ and makes D measurable. More precisely, if $\mathcal{Q}$ refines $\mathcal{P}$ and D is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_D$.*

Proof. Each old atom $B \in \mathcal{P}$ is replaced by the nonempty sets among $B \cap D$ and $B \cap D^c$, so the resulting sets are disjoint, cover Ω, and each lies inside an atom of $\mathcal{P}$. Hence $\mathcal{P}_D$ is a refinement of $\mathcal{P}$. Moreover,

$$D = \bigcup_{B \in \mathcal{P}} (B \cap D),$$

so D is a union of $\mathcal{P}_D$-atoms. Now let $\mathcal{Q}$ refine $\mathcal{P}$ and make D measurable. For every $Q \in \mathcal{Q}$, there is $B \in \mathcal{P}$ with $Q \subseteq B$, and measurability of D with respect to $\mathcal{Q}$ implies either $Q \subseteq D$ or $Q \subseteq D^c$. Thus Q is contained in either $B \cap D$ or $B \cap D^c$, i.e., in an atom of $\mathcal{P}_D$. Therefore $\mathcal{Q}$ refines $\mathcal{P}_D$. □

The theorem gives a canonical update. The evidence determines the least refined representation: atoms untouched by D remain unchanged, while atoms crossed by D split into the two pieces that the evidence distinguishes.

G.2 The sigma-algebra form

For general spaces, the same operation is the join of sigma-algebras. If the current information is $\mathcal{F}_t$ and the evidence is a variable U_t, the minimal updated information is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(U_t). \tag{7}$$

Definition G.3 (Generated sigma-algebra and join)**.** *For a random variable U, $\sigma(U)$ denotes the smallest sigma-algebra with respect to which U is measurable. For two sigma-algebras $\mathcal{F}$ and $\mathcal{H}$, their join is*

$$\mathcal{F} \vee \mathcal{H} = \sigma(\mathcal{F} \cup \mathcal{H}),$$

the smallest sigma-algebra containing both. In particular, $\mathcal{F} \vee \sigma(U)$ is the information structure obtained by adding observability of U to $\mathcal{F}$.

Theorem G.4 (Minimal sigma-algebra refinement)**.** *Let $\mathcal{F} \subseteq \mathcal{E}$ be a sigma-algebra and let U be a random variable. The join $\mathcal{F} \vee \sigma(U)$ is the smallest sigma-algebra $\mathcal{G}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$-measurable.*

Proof. By definition, $\mathcal{F} \vee \sigma(U) = \sigma(\mathcal{F} \cup \sigma(U))$, so it contains $\mathcal{F}$ and makes U measurable. If another sigma-algebra $\mathcal{G}$ contains $\mathcal{F}$ and makes U measurable, then $\mathcal{F} \cup \sigma(U) \subseteq \mathcal{G}$. Since $\mathcal{G}$ is a sigma-algebra, $\sigma(\mathcal{F} \cup \sigma(U)) \subseteq \mathcal{G}$. □

Thus the same object governs finite decision-tree splits and measurable-space refinement. A distinction is not an auxiliary feature added after learning; it is a change in the sigma-algebra of what the learner may condition on. The obstruction in Section F says when such a change is necessary; minimal refinement says how little information must be added.

H The Value of a Distinction

Minimal refinement says how to add a distinction; it does not license every possible distinction. The value question is task-dependent: when does the new distinction actually change the best achievable prediction? We measure this by the decrease in optimal risk after the information structure is refined, in the same decision-theoretic sense used in classical comparisons of statistical experiments and value of information [Blackwell, 1953, Berger, 1985].

The identities in this section are population Bayes-risk statements and require the relevant expectations, entropies, or second moments to be finite. Bounded-loss assumptions enter only in the empirical concentration and validation results.

Definition H.1 (Distinction value)**.** *For information structures $\mathcal{F} \subseteq \mathcal{G}$, the value of refining $\mathcal{F}$ to $\mathcal{G}$ under loss L is*

$$V_L(\mathcal{F}, \mathcal{G}) = R_L(\mathcal{F}) - R_L(\mathcal{G}).$$

For a binary distinction U, its conditional value at $\mathcal{F}$ is

$$V_L(U \mid \mathcal{F}) = R_L(\mathcal{F}) - R_L(\mathcal{F} \vee \sigma(U)).$$

In the finite partition model, the analogous value is $V_L(U \mid \Pi) = R_L(\Pi) - R_L(\Pi \vee U)$.

H.1 Risk monotonicity

The first fact is purely order-theoretic. If $\mathcal{G}$ contains all events in $\mathcal{F}$, then every $\mathcal{F}$-measurable predictor is also $\mathcal{G}$-measurable. The decision class has only grown; this is the Bayes-risk analogue of Blackwell's monotonicity of decision value under more informative experiments [Blackwell, 1953].

Theorem H.2 (Bayes-risk monotonicity)**.** *If $\mathcal{F} \subseteq \mathcal{G}$, then for every loss L for which both risks are well-defined,*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

Proof. Every $\mathcal{F}$-measurable action is also $\mathcal{G}$-measurable. Taking the infimum of the same expected loss over the larger feasible set gives the claim. □

This is a population statement. It says that information cannot hurt the best possible decision rule. It does not say that a finite implementation should refine without restraint. Statistical estimation, finite samples, and model selection can overfit; those are algorithmic questions, not failures of the information order. This is why the CGA algorithm later uses penalties, thresholds, and sample-split scoring before committing a new concept.

H.2 Loss-relevant refinement

A distinction is useful only if it changes the Bayes-optimal decision. The next theorem is the decision-theoretic core of the paper: zero refinement value means the refined optimum was already expressible under the old information.

Theorem H.3 (Loss-relevant refinement)**.** *Let $\mathcal{F} \subseteq \mathcal{G} \subseteq \mathcal{E}$. Assume Bayes actions exist under $\mathcal{F}$ and $\mathcal{G}$, and that the $\mathcal{G}$-Bayes action is almost surely unique. Then*

$$R_L(\mathcal{F}) - R_L(\mathcal{G}) = 0$$

if and only if $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version.

Proof. If $a^\star_{\mathcal{G}}$ is $\mathcal{F}$-measurable, then it is feasible for $R_L(\mathcal{F})$, so

$$R_L(\mathcal{F}) \leq \mathbb{E}[L(Y, a^\star_{\mathcal{G}})] = R_L(\mathcal{G}).$$

Monotonicity gives the reverse inequality. Conversely, if the risks are equal and $a^\star_{\mathcal{F}}$ is an $\mathcal{F}$-Bayes action, then $a^\star_{\mathcal{F}}$ is also feasible under $\mathcal{G}$ and achieves $R_L(\mathcal{G})$. Almost sure uniqueness under $\mathcal{G}$ forces $a^\star_{\mathcal{F}} = a^\star_{\mathcal{G}}$ almost surely, so $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version. □

H.3 Log loss

For a finite target variable Y, log loss asks the learner to output a conditional distribution. The optimal prediction under $\mathcal{F}$ is the conditional law of Y given $\mathcal{F}$, and the optimal risk is conditional entropy, a standard identity for logarithmic scoring rules and information theory [Cover and Thomas, 2006, Gneiting and Raftery, 2007].

Theorem H.4 (Log-loss value)**.** *Let Y be finite and let the loss be $-\log q(Y)$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Here $I(Y; \mathcal{G} \mid \mathcal{F})$ denotes $H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G})$. Writing

$$p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F}), \qquad p_{\mathcal{G}}(y) = \mathbb{P}(Y = y \mid \mathcal{G}),$$

the same identity has the conditional-KL form

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = \mathbb{E}\left[D_{\mathrm{KL}}(p_{\mathcal{G}}(\cdot) \| p_{\mathcal{F}}(\cdot))\right]. \tag{8}$$

Thus, for classification and language modeling, a refinement is useful exactly when it changes the conditional consequence distribution with positive probability. If the new bit does not tell us anything about the next consequence beyond the old information, it is a gratuitous distinction, even if it separates inputs.

Corollary H.5 (Binary distinction value)**.** *Let $U \in \{0, 1\}$ be a candidate distinction and let $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Under log loss,*

$$\begin{aligned} R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) &= I(Y; U \mid \mathcal{F}) \\ &= \mathbb{E}\left[\sum_{u \in \{0,1\}} \mathbb{P}(U = u \mid \mathcal{F}) D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}, U = u) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right]. \end{aligned}$$

H.4 Squared loss

For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss, the optimal predictor under $\mathcal{F}$ is $\mathbb{E}[Y \mid \mathcal{F}]$, the L^2 projection onto the closed subspace of $\mathcal{F}$-measurable random variables [Billingsley, 1995, Kallenberg, 2002]. Refinement helps precisely when it changes the conditional mean.

Theorem H.6 (Squared-loss value)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

The identity is the Hilbert-space analogue of the log-loss result. A distinction has squared-loss value when the cells it creates have different conditional means. In regression language, refinement is worth paying for when it separates consequences, not merely when it separates inputs.

H.5 Minimal loss-sufficient refinement

Minimal refinement in Section G adds the least information needed to observe a proposed variable. The loss-sufficient question is slightly different: among the refinements inside $\mathcal{G}$, what is the least information needed to attain the $\mathcal{G}$-Bayes risk?

Theorem H.7 (Minimal loss-sufficient refinement)**.** *Work in the measurable-action convention of Section D. Let $\mathcal{F} \subseteq \mathcal{G}$ and assume the $\mathcal{G}$-Bayes action $a^\star_{\mathcal{G}} : \Omega \to \mathsf{A}$ exists as a $(\mathcal{G}, \Sigma_{\mathsf{A}})$-measurable map and is almost surely unique. Also assume the Bayes infimum is attained for every intermediate sigma-algebra $\mathcal{H}$ with $\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}$ that is considered below. Up to null sets, the smallest sigma-algebra $\mathcal{H}$ satisfying*

$$\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}, \qquad R_L(\mathcal{H}) = R_L(\mathcal{G})$$

is

$$\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}}).$$

For finite Y under log loss, the corresponding smallest sufficient refinement is

$$\mathcal{F} \vee \sigma(p_{\mathcal{G}}), \qquad p_{\mathcal{G}}(\cdot) = \mathbb{P}(Y \in \cdot \mid \mathcal{G}).$$

Proof. The action $a^\star_{\mathcal{G}}$ is measurable with respect to $\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}})$, so this refinement attains $R_L(\mathcal{G})$. If any $\mathcal{H}$ with $\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}$ also attains $R_L(\mathcal{G})$, Theorem H.3 applied to $\mathcal{H} \subseteq \mathcal{G}$ implies that $a^\star_{\mathcal{G}}$ has an $\mathcal{H}$-measurable version. Hence $\mathcal{F} \vee \sigma(a^\star_{\mathcal{G}}) \subseteq \mathcal{H}$ up to null sets. Under log loss, the Bayes action is the conditional law $p_{\mathcal{G}}$, giving the final statement. □

I Empirical Split Scores and Finite-Sample Refinement

Population refinement value becomes empirical information gain or variance reduction, but finite samples require penalties, margins, and held-out validation.

I.1 Finite empirical scores

Definition I.1 (Empirical distinction scores)**.** *Suppose an atom B contains samples (x_j, y_j) and a candidate distinction $g : B \to \{0,1\}$ splits it into B_0 and B_1. The formulas below are evaluated for nontrivial empirical splits with $|B_0|, |B_1| > 0$. A candidate with an empty empirical child is assigned score zero and is not accepted by a positive-gain split rule.*

For classification, let $\hat{p}_B$, $\hat{p}_{B_0}$, and $\hat{p}_{B_1}$ be empirical label distributions. The empirical log-loss improvement is

$$\Delta_{\log}(B, g) = |B|\, H(\hat{p}_B) - |B_0|\, H(\hat{p}_{B_0}) - |B_1|\, H(\hat{p}_{B_1}).$$

Equivalently,

$$\Delta_{\log}(B, g) = |B|\, \widehat{I}(Y; g \mid B).$$

The normalized empirical edge score is therefore

$$\widehat{I}(Y; g \mid B) = H(\hat{p}_B) - \sum_{b \in \{0,1\}} \frac{|B_b|}{|B|} H(\hat{p}_{B_b}).$$

For regression, define

$$\hat{\mu}_B = \frac{1}{|B|} \sum_{j \in B} y_j, \qquad \mathrm{SSE}(B) = \sum_{j \in B} \|y_j - \hat{\mu}_B\|_2^2 .$$

The empirical squared-loss improvement is

$$\Delta_2(B, g) = \mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1).$$

It also has the between-cell form

$$\Delta_2(B, g) = |B_0|\, \|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1|\, \|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2 .$$

The population identities lead directly to these standard empirical split scores: information gain for classification and variance reduction for regression trees [Quinlan, 1986, Breiman et al., 1984, Domingos and Hulten, 2000]. They implement the same principle as Definition H.1: split only when the proposed distinction has measurable consequence value. The CGA algorithm below turns this population rule into a guarded empirical decision.

I.2 Distinction complexity

Definition I.2 (Distinction complexity)**.** *Let Π be an initial partition, let $\Pi^\star$ be a target refinement, and let $\mathcal{U} \subseteq \{0,1\}^\Omega$ be a distinction class. For $\epsilon \geq 0$, the loss-dependent distinction complexity is*

$$d_L^\epsilon(\Pi, \Pi^\star; \mathcal{U}) = \min \left\{ m : \exists U_1, \ldots, U_m \in \mathcal{U} \text{ such that } R_L(\Pi \vee U_1 \vee \cdots \vee U_m) \leq R_L(\Pi^\star) + \epsilon \right\}, \tag{9}$$

with value $+\infty$ if no such sequence exists.

The refinement gap measures the value of a target information structure. Distinction complexity measures how many admissible bits are needed to reach it. Repair-only learning is the case $m = 0$. Decision trees, graph refinements, and prototype split systems differ partly in the distinction classes and ordering constraints they impose. CGA corresponds to a graph-generated class of local binary predicates proposed from confused runtime states.

I.3 Greedy recovery under realizability

The next theorem is a clean realizable upper bound for greedy refinement. It is intentionally stated for the abstract partition model rather than for Soft-CGA retrieval. The distinction class is understood to contain only prediction-time observable distinctions, as in Definition E.2.

Definition I.3 (Target-respecting distinction)**.** *Let $\Pi^\star$ be a target partition. A distinction $U : \Omega \to \{0,1\}$ is $\Pi^\star$-respecting if it does not split any atom of $\Pi^\star$: for every $C \in \Pi^\star$, either $C \subseteq U^{-1}(0)$ or $C \subseteq U^{-1}(1)$.*

Definition I.4 (Impure atom and local split value)**.** *For an atom $B \in \Pi$, write $\Pi^\star|_B = \{C \in \Pi^\star : C \cap B \neq \varnothing\}$. The atom B is impure relative to $\Pi^\star$ if $|\Pi^\star|_B| > 1$. For a candidate distinction U, its local value on B is*

$$V_L(B,U) = R_L(\{B\}) - R_L(\{B\} \vee U)\,,$$

where the risks are conditional on $\omega \in B$. Under log loss, $V_{\log}(B,U) = I(Y;U \mid \omega \in B)$.

Theorem I.5 (Greedy recovery under target-respecting splits)**.** *Let*

$$K^+ = |\{C \in \Pi^\star : P(C) > 0\}|\,, \qquad m_0^+ = |\{B \in \Pi_0 : P(B) > 0\}|\,.$$

Suppose the current partition Π_0 is initially a coarsening of $\Pi^\star$. Assume that every positive-mass impure current atom B admits a $\Pi^\star$-respecting distinction $U \in \mathcal{U}$ such that U splits $\Pi^\star|_B$ into two groups whose unions both have positive conditional probability under $P(\cdot \mid B)$ and

$$V_L(B,U) \geq \gamma.$$

Consider a greedy refinement algorithm that repeatedly selects a positive-mass impure current atom whenever one exists. Assume its candidate set contains at least one such distinction for every selected impure atom, its accepted candidates are $\Pi^\star$-respecting, and its empirical local values satisfy

$$\left|\widehat{V}_L(B,U) - V_L(B,U)\right| \leq \gamma/4$$

for all candidates it scores. If the algorithm accepts a split whenever $\widehat{V}_L(B,U) > \gamma/2$, then it accepts a useful split for every selected impure atom and recovers $\Pi^\star$ on the positive-mass support after at most $K^+ - m_0^+ \leq K^+ - 1$ accepted splits.

Proof. Because every accepted split is $\Pi^\star$-respecting, the current partition remains a coarsening of $\Pi^\star$ after each accepted refinement. A candidate with true value at least γ has empirical value at least $3\gamma/4$, so it exceeds the $\gamma/2$ threshold and is accepted when proposed. Conversely, any accepted candidate has true value greater than $\gamma/4$ by the uniform estimation event. A $\Pi^\star$-respecting split with positive value cannot be trivial on the selected atom: if one side had zero conditional mass, the conditional Bayes risk would be unchanged. Thus any accepted candidate separates the selected impure atom into two positive-mass unions of target atoms. It replaces one positive-mass current atom by two positive-mass unions of target atoms and increases the number of represented positive-mass current atoms by one. The algorithm keeps selecting impure positive-mass atoms until none remain. Starting from m_0^+ positive-mass current atoms and ending with at most K^+ positive-mass target atoms, at most $K^+ - m_0^+ \leq K^+ - 1$ accepted splits are needed. □

For finite $\mathcal{U}$, finite action class A, and bounded loss, the event in Theorem I.5 follows from a standard held-out union bound over scored split-action pairs [Hoeffding, 1963, Devroye et al., 1996]. It suffices to score each accepted split with

$$m = O\left(\frac{1}{\gamma^2}\log\frac{K^+\,|\mathcal{U}|\,|\mathsf{A}|^3}{\delta}\right)$$

independent validation samples, up to constants and the bounded-loss range. For finite-label log-loss split scores or regression scores with infinite action spaces, the same conclusion requires the corresponding entropy, variance, or empirical-risk concentration assumption; bounded loss and finite $\mathcal{U}$ alone do not control the Bayes-action estimation error.

I.4 A finite-class oracle inequality

The same finite model gives a structural-risk bound for searching over refinements. Let $\mathfrak{P}_B(\mathcal{U})$ be the set of partitions obtainable from an initial partition Π_0 by at most B global distinctions from finite $\mathcal{U}$, where each distinction is applied by the join operation in (2). If Π_0 has m_0 atoms, every $\Pi \in \mathfrak{P}_B(\mathcal{U})$ has at most $m_0 2^B$ atoms, because one binary distinction can cut every current atom. The sharper $m_0 + B$ count applies only to a different local-split class in which each step is restricted to split a single current atom.

Theorem I.6 (Finite-class refinement oracle inequality)**.** *Assume $L \in [0,1]$ and A is finite. Let $\widehat{R}_L(\Pi)$ be the empirical risk of the best atomwise action for Π on n iid validation samples, and let*

$$\widehat{\Pi} \in \operatorname*{argmin}_{\Pi \in \mathfrak{P}_B(\mathcal{U})} \left\{ \widehat{R}_L(\Pi) + \lambda \left|\Pi\right| \right\}.$$

Then with probability at least $1-\delta$,

$$R_L(\widehat{\Pi}) + \lambda \left|\widehat{\Pi}\right| \leq \min_{\Pi \in \mathfrak{P}_B(\mathcal{U})} \left\{ R_L(\Pi) + \lambda \left|\Pi\right| \right\} + 2 \left(\frac{B \log |\mathcal{U}| + m_0 2^B \log |\mathsf{A}| + \log(B+1) + \log(2/\delta)}{2n} \right)^{1/2}.$$

Proof. For any fixed sequence of at most B distinctions and any assignment of actions to the resulting at most $m_0 2^B$ atoms, Hoeffding's inequality controls the empirical-population risk deviation. The number of sequence lengths is at most $B+1$, the number of distinction sequences is at most $|\mathcal{U}|^B$, and the number of action assignments for any resulting partition is at most $|\mathsf{A}|^{m_0 2^B}$. Thus the number of partition-action pairs is at most $(B+1)\,|\mathcal{U}|^B\,|\mathsf{A}|^{m_0 2^B}$. A union bound gives a uniform deviation ϵ equal to the square-root term without the leading factor 2. On this event, empirical penalized risk and population penalized risk differ by at most ϵ for every candidate. Applying the usual empirical-minimizer argument gives the displayed 2ϵ bound. □

Theorem I.7 (Local-split refinement oracle inequality)**.** *Let $\mathfrak{P}_B^{\mathrm{loc}}(\mathcal{U})$ be the set of partitions reachable from an initial partition Π_0 with m_0 atoms by at most B local splits, where each split chooses one current atom and one distinction $U \in \mathcal{U}$ and applies the split only inside that atom. Assume $L \in [0,1]$ and A is finite. Let $\widehat{\Pi}$ minimize empirical penalized risk $\widehat{R}_L(\Pi) + \lambda|\Pi|$ over $\mathfrak{P}_B^{\mathrm{loc}}(\mathcal{U})$ on n iid validation samples. Then, with probability at least $1-\delta$,*

$$R_L(\widehat{\Pi}) + \lambda|\widehat{\Pi}| \leq \min_{\Pi \in \mathfrak{P}_B^{\mathrm{loc}}(\mathcal{U})} \{R_L(\Pi) + \lambda|\Pi|\} + 2\sqrt{\frac{B \log |\mathcal{U}| + B \log(m_0 + B) + (m_0 + B) \log |\mathsf{A}| + \log(2(B+1)/\delta)}{2n}}.$$

Proof. Every local-split partition has at most $m_0 + B$ atoms. For a sequence length at most B, there are at most $|\mathcal{U}|^B$ choices of distinctions and at most $(m_0 + B)^B$ choices of current atoms to split, since the number of atoms never exceeds $m_0 + B$. For any resulting partition, the number of atomwise action assignments is at most $|\mathsf{A}|^{m_0+B}$. Hoeffding's inequality and a union bound over sequence lengths, local split sequences, and action assignments give a uniform deviation with the square-root term without the leading factor 2. The empirical-minimizer argument then adds the second copy of the deviation. □

J Concept Graph Attention: A Data Structure for Refinable Partitions

CGA implements refinable information structures as a graph: edges generate distinctions and nodes store repairable memories. The ingredients are related to decision-tree refinement, adaptive category creation, prototype memories, and external neural memories, but CGA uses them to implement the repair/refinement distinction [Breiman et al., 1984, Domingos and Hulten, 2000, Carpenter and Grossberg, 1987b, Grossberg, 2013, Kohonen, 1990, Weston et al., 2015, Graves et al., 2016]. The theory so far gives a diagnostic: decide whether an observed error is better explained by Δ_{repair}

at fixed information or by Δ_{refine} from missing distinctions. A runtime implementation therefore needs a state representation that can both repair local statistics and add distinctions. CGA is the data-structure realization of the abstract refinable-partition model.

J.1 Why a graph, not a tree?

Definition J.1 (CGA concept)**.** *A CGA concept is a memory location selected by a graph-generated hard-route map and equipped with a local memory. It is not claimed to be a human-interpretable semantic concept and need not correspond one-to-one with an atom of the edge partition. Local memories live at graph nodes, and distinction predicates live on edges between nodes that have been confused.*

The central object is

$$\text{CGA} = \text{graph-generated information structure} + \text{local memory repair}.$$

The graph is not metadata: its edges generate a finite edge-skeleton information structure and gate the read path, while anchor order completes the exact hard route. The partition and generated-sigma-algebra language follows standard finite measurable-space constructions [Billingsley, 1995, Kallenberg, 2002]. The main-text statements below give the operational theorem forms; full proofs are in Appendix Q.

Definition J.2 (Hard-CGA and Soft-CGA)**.** *Hard-CGA is the formal algorithm: fixed anchors and accepted binary edges generate an edge skeleton, while the exact hard route is measured by the join of this edge skeleton with the anchor-distance ordering. A repair changes only local actions or memories and leaves the hard-route information fixed; a refinement adds an anchored concept, new edge generators, and the anchor-order events involving the new anchor. Soft-CGA is the practical relaxation used for sparse neural hidden states: movable centroids retrieve candidates, smooth edge gates approximate hard eligibility, and routing weights mix local memories.*

The partition-level guarantees are exact for Hard-CGA. Soft-CGA is covered only as an approximation under the candidate-recall and margin conditions in Theorem J.9.

J.2 Hard-CGA state and invariants

Definition J.3 (Hard-CGA state and anchor edge)**.** *Let $\phi_\theta : \mathcal{X} \to \mathbb{R}^d$ be an encoder, write $h = \phi_\theta(x)$, and let $\mathsf{H} \subseteq \mathbb{R}^d$ denote the hidden-state space on which the runtime structure is evaluated. A Concept Graph Attention has runtime state*

$$\mathcal{S}_t = \left(\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t\right), \qquad G_t = (V_t, E_t), \qquad V_t = \{1, \dots, N_t\}.$$

Each component has a role forced by the refine-or-repair split. A runtime concept needs a stable identity, so $a_j \in \mathbb{R}^d$ is a frozen distinction anchor. Retrieval may adapt as examples arrive, so $c_j \in \mathbb{R}^d$ is a movable centroid used only by the soft implementation. State inside an adequate concept must be repairable, so M_j is a local memory and n_j is a usage count. If a concept is created because it was confused with existing concepts, that conflict becomes an edge in E_t. A new concept is initialized with $a_j = c_j = h$. Subsequent repair may move c_j, but anchors remain fixed once created, so the hard information structure is unchanged by centroid repair.

An edge $e = (i, j)$ stores a linear distinction

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j),$$

with side labels $s_{e,i} = +1$ and $s_{e,j} = -1$. Its binary generator is

$$g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}.$$

Definition J.4 (Graph signature and edge-generated partition)**.** *The edge set induces the* edge skeleton *information structure on* H*. The graph signature is*

$$\sigma_t^G(h) = \big(g_e(h)\big)_{e \in E_t} \in \{0,1\}^{E_t}.$$

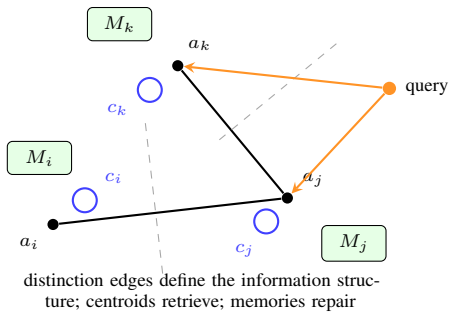

Figure 3: CGA state. Anchors are stable concept identities, centroids are movable retrieval statistics, distinction edges generate the information structure, and concept memories are repaired after a concept is judged adequate.

Two hidden states are indistinguishable by the current graph if their signatures agree. The nonempty level sets

$$A_a^t = \{h \in \mathsf{H} : \sigma_t^G(h) = a\}, \qquad a \in \{0,1\}^{E_t},$$

form a finite partition $\mathcal{P}_t^G$ of hidden-state space, and the generated sigma-algebra is $\sigma(\mathcal{P}_t^G)$. Adding a new edge is therefore a join with one binary generator:

$$\mathcal{P}_{t+1}^G = \mathcal{P}_t^G \vee \sigma(g_e).$$

This edge partition is only the skeleton. The exact hard route also needs the ordering of eligible anchors by anchor distance; that order is added below as $\mathcal{F}_t^{\text{anchor-order}}$. Centroids and local memories are adaptive statistics attached to hard-route states or sparse neighborhoods, not generators of the exact Hard-CGA information structure. Concept nodes and graph-generated partition atoms are related but not identical: a node is a data-structure representative with memory and retrieval statistics; the formal information structure is generated by edge signatures and anchor-order events.

Theorem J.5 (Anchor distinction separates its endpoints)**.** *If $a_i \neq a_j$ and u_e, b_e are defined as above for edge $e = (i,j)$, then*

$$u_e^\top a_i - b_e = \frac{1}{2}\left\|a_i - a_j\right\|_2, \qquad u_e^\top a_j - b_e = -\frac{1}{2}\left\|a_i - a_j\right\|_2.$$

Thus each edge distinction separates the two anchors that caused it.

Implementations should create such an edge only when the anchor separation clears a numerical guard,

$$\left\|a_i - a_j\right\|_2 > \eta_{\text{anchor}},$$

for a fixed $\eta_{\text{anchor}} > 0$. If a proposed new anchor is duplicate or nearly duplicate with a confused neighbor, the algorithm should skip that edge and fall back to repair, merge, or another admissible distinction.

Theorem J.6 (CGA graph generates a finite information structure)**.** *For any finite edge set E_t, equality of signatures $\sigma_t^G(h)$ is an equivalence relation whose classes form a finite edge-skeleton partition $\mathcal{P}_t^G$. Adding one edge g refines this skeleton by $\mathcal{P}_t^G \vee \sigma(g)$.*

J.3 Soft-CGA edge-gated prediction

The hard route selects the nearest eligible anchor using fixed anchors, accepted edges, and anchor-distance order, so repairs that change only memories and centroids do not change $\mathcal{F}_t^{\text{CGA}}$. Given h, Soft-CGA instead computes retrieval distances $d_j(h) = \|h - c_j\|_2^2$ to movable centroids. Moving centroids can change the smooth routing weights and therefore the actual Soft-CGA predictor, even when the Hard-CGA skeleton is fixed. Thus centroid repair is a repair in the formal hard-route structure, not a strict claim that the soft read map remains partition-measurable. A centroid-only top-k set is not sufficient for the formal hard-route guarantee: if the hard eligible anchor never

enters the candidate set, edge gates cannot recover it. Approximate-nearest-neighbor systems can make the retrieval term sublinear or hardware-efficient, but the formal guarantee still needs recall of the hard-route candidate [Indyk and Motwani, 1998, Johnson et al., 2019]. We therefore use a recall-oriented candidate set

$$N(h) = N_{k_c}^{\text{centroid}}(h) \cup N_{k_a}^{\text{anchor}}(h) \cup \text{GraphNeighbors}\big(N_{k_c}^{\text{centroid}}(h)\big)\,,$$

with duplicates removed and an optional budget cap after the union. The first term gives adaptive locality, the second protects the anchor-defined hard route, and the third recovers concepts connected by recent consequence conflicts. In exact small-scale implementations one may take $N(h) = \{1, \ldots, N_t\}$. If $\text{Inc}_t(j)$ is the set of edges incident to concept j, define the soft compatibility of h with concept j by

$$\log \chi_j(h) = \frac{1}{\sqrt{\max\{1, |\text{Inc}_t(j)|\}}} \sum_{e \in \text{Inc}_t(j)} \beta_e \log \text{sigmoid}\left(\frac{s_{e,j}(u_e^\top h - b_e)}{\tau_e}\right),$$

where structural distinction edges have $\beta_e \in (0, 1]$ and the empty sum gives $\chi_j(h) = 1$. The degree normalization avoids systematically suppressing high-degree concepts while preserving hard rejection in the low-temperature limit. The edge-gated routing weights are

$$\alpha_j(h) = \frac{\exp(-d_j(h)/T)\chi_j(h)}{\sum_{r \in N(h)} \exp(-d_r(h)/T)\chi_r(h)}, \qquad j \in N(h). \tag{10}$$

The prediction is the local mixture

$$\hat{y} = \sum_{j \in N(h)} \alpha_j(h) M_j(h).$$

The hard route makes the information-structure connection explicit. A concept j is eligible for h if every incident edge bit places h on the side recorded for j:

$$\begin{cases} g_e(h) = 1, & s_{e,j} = +1, \\ g_e(h) = 0, & s_{e,j} = -1, \end{cases} \qquad \text{for every } e \in \text{Inc}_t(j).$$

This convention assigns boundary points to the nonpositive side because $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$. The exact hard route used in the refinement theory routes to the nearest eligible anchor, falling back to the nearest anchor if no eligible concept is available. The soft read in (10) is the trainable implementation: it may retrieve by movable centroids, but those centroids are retrieval statistics rather than formal information cells.

The distinction partition alone does not determine the whole exact hard route, because eligible concepts still need to be ordered. Anchor distances provide this stable order geometry. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be the finite sigma-algebra generated by all pairwise anchor-distance comparison events

$$\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\}, \qquad 1 \le i, j \le N_t,$$

with deterministic tie-breaking. The hard routing skeleton is measurable with respect to

$$\mathcal{F}_t^{\text{CGA}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}.$$

The exact CGA route map can be written as

$$I_{\mathcal{S}_t}(h) = \left((g_e(h))_{e \in E_t}, \text{HardRoute}_{\mathcal{S}_t}(h)\right).$$

This map is measurable with respect to $\mathcal{F}_t^{\text{CGA}}$, so

$$\sigma(I_{\mathcal{S}_t}) \subseteq \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}.$$

The inclusion can be strict: the full anchor-order sigma-algebra records pairwise order comparisons that may not affect the final selected concept. One may use $\mathcal{F}_t^{\text{CGA}}$ as a stable upper information structure with simple repair-stability proofs, or use the coarser exact-route sigma-algebra $\sigma(I_{\mathcal{S}_t})$ when the selected memory location itself is the desired state. Nodes are memory locations selected by this measurable route map; they need not be in one-to-one correspondence with atoms of the edge-skeleton partition. The soft read keeps the same anchored graph skeleton but may use movable centroids and smooth gates for locality; repairs can therefore change its effective smooth read geometry. Soft-CGA should be read as a differentiable approximation to the anchor-generated hard information map, with exact partition claims recovered only in the hard-route limit or under explicit approximation assumptions.

Theorem J.7 (Anchor-based hard routing is graph-plus-anchor-order measurable)**.** *With finite E_t, finite frozen anchors $a_1, \ldots, a_{N_t}$, and deterministic tie-breaking, the exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\mathrm{CGA}} = \mathcal{F}_t^{\mathrm{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.*

The next statement is analytic rather than structural: it applies to whichever fixed distances a soft read uses. With anchor distances it recovers the exact hard route; with centroid distances it describes the behavior of the soft retrieval approximation. It also exposes the candidate-recall requirement that links the formal hard route to the sparse implementation.

Condition J.8 (Candidate recall)**.** *Let $j^\star(h)$ be the unique concept selected by the exact hard route for query h. The sparse soft read satisfies candidate recall at h if*

$$j^\star(h) \in N(h).$$

If this condition fails, the soft read has no guarantee of selecting the hard route, regardless of how sharp the edge gates are.

Theorem J.9 (Soft routing approaches hard routing)**.** *Assume candidate recall holds for h, and let $j^\star$ be the recalled hard-route concept. Split the remaining candidates into eligible competitors, which satisfy the hard structural constraints, and wrong-side competitors, which violate at least one structural edge. Eligible competitors are removed by distance: if $j^\star$ has distance margin $\delta > 0$ over every eligible competitor and the eligible/reference compatibility ratios are uniformly bounded, their total soft mass decays exponentially as $T \to 0$. Wrong-side competitors are removed by edge gates: if every wrong-side candidate violates some incident structural edge with weight at least $\beta_{\min} > 0$, signed margin at least $\delta_e > 0$, and degree at most $D_{\max}$, then its compatibility decays uniformly after degree normalization. The hard-route limit is therefore an ordered or controlled joint limit: edge temperature must suppress wrong-side candidates before distance temperature makes nearest-neighbor selection decisive. Equivalently, either the wrong-side/reference distance-score ratios remain uniformly bounded, or for distance advantage Δ_d of wrong-side candidates over an eligible reference,*

$$\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e} - \frac{\Delta_d}{T} \to +\infty.$$

Under either condition, total wrong-side routing mass vanishes. Together with the distance margin for eligible competitors, this gives convergence to the hard route. Without candidate recall, the theorem gives no correctness guarantee for an omitted hard-route concept.

Algorithm 2 CGA-PREDICT

$\mathcal{S}_t = (\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t), \quad x \in \mathcal{X}$

1: $h \leftarrow \phi_\theta(x)$

2: $N(h) \leftarrow N_{k_c}^{\mathrm{centroid}}(h) \cup N_{k_a}^{\mathrm{anchor}}(h) \cup \mathrm{GraphNeighbors}(N_{k_c}^{\mathrm{centroid}}(h))$

3: $\chi_j(h) \leftarrow \exp\left\{ \frac{1}{\sqrt{\max\{1, |\mathrm{Inc}_t(j)|\}}} \sum_{e \in \mathrm{Inc}_t(j)} \beta_e \log \mathrm{sigmoid}\big(s_{e,j}(u_e^\top h - b_e)/\tau_e\big) \right\}, \quad j \in N(h)$

4: $\alpha_j(h) \leftarrow \dfrac{\exp(-\|h - c_j\|_2^2/T)\chi_j(h)}{\sum_{r \in N(h)} \exp(-\|h - c_r\|_2^2/T)\chi_r(h)}, \quad j \in N(h)$

5: $\hat{a}_t(h) \leftarrow \sum_{j \in N(h)} \alpha_j(h) M_j(h)$

6: **return** $\hat{a}_t(h)$

J.4 Refine or repair on a graph

After observing consequence y, CGA scores empirical versions of the two gaps in Theorem F.2. A repair candidate keeps the CGA information map fixed and updates only local state. A refinement candidate changes the information map by adding an anchored concept and distinction edges. The committed operation is the one whose held-out gain justifies its cost and safety margin.

Let $j^\star$ be the highest-weight concept under (10). Define local surprise

$$\rho_t = L(y, M_{j^\star}(h)).$$

Define neighborhood consequence disagreement on $N(h)$ by the task-appropriate local prediction divergence. For classification one may use

$$D_t = \max_{i,r \in N(h)} \mathrm{TV}(p_i(\cdot \mid h), p_r(\cdot \mid h)),$$

where p_j is the distribution represented by memory M_j. For regression one may use

$$D_t = \max_{i,r \in N(h)} \|\mu_i(h) - \mu_r(h)\|_2 ,$$

and for language modeling the log-loss surprise $-\log p_{j^\star}(x_{t+1} \mid x_{\leq t})$ is the local disagreement signal. These quantities are proposal signals. They decide which candidates are worth scoring; they do not define repair or refinement.

All gap estimates below are computed on a held-out scoring buffer $\mathcal{B}_t^{\mathrm{score}}$. Refinement proposals may use a disjoint proposal buffer $\mathcal{B}_t^{\mathrm{prop}}$ together with the current observation; the scoring buffer is held out from proposal generation and from previous accept/reject decisions that determine the current state. Theorem J.12 applies when each scoring decision uses fresh held-out samples, or samples not previously used to choose proposals or update the state. Reservoir sampling can maintain disjoint proposal and scoring streams, but reusing the same validation buffer adaptively requires reusable-holdout or adaptive-validation control [Dwork et al., 2015b,a]. Without that extra control, the empirical rule is a heuristic rather than the Hoeffding guarantee.

The edges created by a refinement are attached to a concrete confused-neighbor set. Let $\mathrm{Div}_{\mathcal{Y}}(M_i, M_r; h)$ denote a task-appropriate consequence divergence, such as total variation for classifiers, Euclidean mean distance for regression memories, or next-token distribution divergence for language models. Define

$$\mathrm{Confused}(h, y) = \{i \in N(h) : L(y, M_i(h)) > \tau_{\mathrm{conflict}} \text{ or } \mathrm{Div}_{\mathcal{Y}}(M_i, M_{j^\star}; h) > \tau_{\mathrm{disagree}}\} .$$

When a budget of at most q new edges is used, CGA takes $\mathrm{Confused}_q(h, y)$ to be the q highest-scoring members of this set under

$$\mathrm{score}_i(h, y) = \alpha_i(h) L(y, M_i(h)) + \gamma_{\mathrm{div}} \mathrm{Div}_{\mathcal{Y}}(M_i, M_{j^\star}; h).$$

If the set is empty but refinement is otherwise accepted, the implementation falls back to the highest-score candidates in $N(h) \setminus \{j^\star\}$. Thus "confused neighbors" denotes a concrete operator rather than an informal phrase.

Repair is guarded rather than blindly committed. The implementation proposes a small set of repair candidates, such as a memory-only update, a memory-plus-centroid update, and one or more reduced centroid steps:

$$\mathcal{C}_{\mathrm{repair}}(\mathcal{S}_t, h, y) = \{\mathcal{S}_t^{\mathrm{mem}}, \mathcal{S}_t^{\mathrm{mem+centroid}}, \mathcal{S}_t^{\mathrm{shrink}}, \mathcal{S}_t\}.$$

Each candidate keeps anchors and edges fixed, so it preserves $I_{\mathcal{S}_t}$ and $\mathcal{F}_t^{\mathrm{CGA}}$. On the scoring buffer, the empirical repair gain is

$$\widehat{\Delta}_{\mathrm{repair}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\mathrm{score}}) - \min_{\mathcal{S}' \in \mathcal{C}_{\mathrm{repair}}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\mathrm{score}}). \tag{11}$$

CGA's guarded repair candidate is

$$\mathcal{S}_{t+1} \in \operatorname*{argmin}_{\mathcal{S}' \in \mathcal{C}_{\mathrm{repair}}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\mathrm{score}}) \quad \text{subject to} \quad \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\mathrm{score}}) \leq \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\mathrm{score}}). \tag{12}$$

The identity state is in $\mathcal{C}_{\mathrm{repair}}$, so $\widehat{\Delta}_{\mathrm{repair}} \geq 0$. Thus monotone repair is an enforced algorithmic property, not merely a theorem assumption. The anchor $a_{j^\star}$ is never moved by repair. Consequently, local memory repair and centroid adaptation do not rewrite previously accepted edge generators.

For a triggered refinement, define the candidate index $j_{\mathrm{new}} = N_t + 1$ and the proposed edge set

$$E_{\mathrm{new}}(h, y) = \{(j_{\mathrm{new}}, i) : i \in \mathrm{Confused}_q(h, y),\ \|h - a_i\|_2 > \eta_{\mathrm{anchor}}\} . \tag{13}$$

The corresponding candidate state creates

$$a_{j_{\mathrm{new}}} = h, \qquad c_{j_{\mathrm{new}}} = h, \qquad M_{j_{\mathrm{new}}} = \mathrm{InitMemory}(h, y),$$

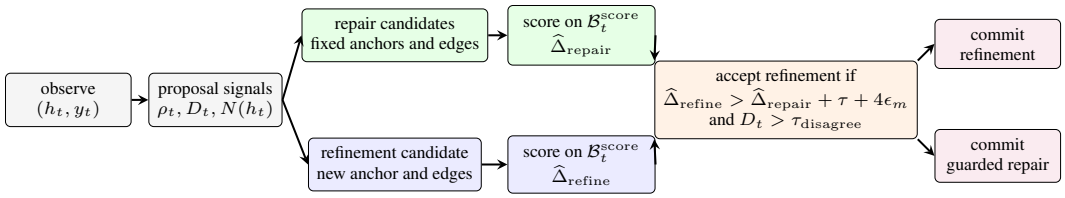

Figure 4: CGA refine-or-repair decision. Proposal signals only choose candidates to score; the committed update is selected by held-out empirical gains. A refinement changes the information structure only when its penalized gain beats the best non-worsening repair by the safety margin; otherwise CGA repairs local state while preserving anchors and edges.

with edge set $E_t \cup E_{\text{new}}(h, y)$. Let $\mathcal{S}_t^{\text{new}}$ denote this state, and let $\widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}})$ be the empirical local loss on the scoring buffer after routing with state $\mathcal{S}$. The empirical penalized refinement gain is

$$\widehat{\Delta}_{\text{refine}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_t^{\text{new}}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} \left|E_{\text{new}}(h, y)\right|. \tag{14}$$

Here λ_{concept} is the penalty for creating the anchored concept and its induced anchor-order bookkeeping, while $\lambda_{\text{edge}} \left|E_{\text{new}}(h, y)\right|$ accounts for the accepted edge generators. Using the edge budget q instead of the actual $|E_{\text{new}}(h, y)|$ is a conservative implementation choice that only makes the acceptance test stricter. The candidate is accepted only if it beats the repair gain by a safety margin:

$$\widehat{\Delta}_{\text{refine}} > \max\{\widehat{\Delta}_{\text{repair}}, 0\} + \tau + 4\epsilon_m \qquad \text{and} \qquad D_t > \tau_{\text{disagree}}. \tag{15}$$

Here ϵ_m is a statistical safety margin for a scoring buffer of size $m = |\mathcal{B}_t^{\text{score}}|$; it can be set to zero for a purely empirical implementation, or to the adaptive validation margin in Theorem J.12. If the refinement test fails, CGA commits the guarded repair candidate in (12). This makes refinement a statistically guarded gap comparison rather than an unscored growth heuristic.

Algorithm 3 CGA-OBSERVE

$\mathcal{S}_t, \quad (x_t, y_t), \quad \mathcal{B}_t^{\text{prop}}, \quad \mathcal{B}_t^{\text{score}}$
$h_t \leftarrow \phi_\theta(x_t)$
if $N_t = 0$ **then**
  $\mathcal{S}_{t+1} \leftarrow (\{(h_t, h_t, \text{InitMemory}(h_t, y_t), 1)\}, (\{1\}, \varnothing))$
  return $\mathcal{S}_{t+1}$
end if
$\mathcal{N}_t \leftarrow N_{k_c}^{\text{centroid}}(h_t) \cup N_{k_a}^{\text{anchor}}(h_t) \cup \text{GraphNeighbors}(N_{k_c}^{\text{centroid}}(h_t))$
$\alpha_{t,j} \leftarrow \alpha_j(h_t), \quad j \in \mathcal{N}_t$
$j_t^\star \leftarrow \text{argmax}_{j \in \mathcal{N}_t} \alpha_{t,j}$
$\rho_t \leftarrow L(y_t, M_{j_t^\star}(h_t))$
$D_t \leftarrow \max_{i,r \in \mathcal{N}_t} \text{Div}_{\mathcal{Y}}(M_i, M_r; h_t)$
$\mathcal{C}_t^{\text{repair}} \leftarrow \mathcal{C}_{\text{repair}}(\mathcal{S}_t, h_t, y_t)$
$\widehat{\Delta}_t^{\text{repair}} \leftarrow \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \min_{\mathcal{S}' \in \mathcal{C}_t^{\text{repair}}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}})$
$\mathcal{S}_t^{\text{repair}} \in \text{argmin}_{\substack{\mathcal{S}' \in \mathcal{C}_t^{\text{repair}} \\ \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}}) \le \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}})}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\text{score}})$
$z_t \leftarrow \mathbf{1}\{\rho_t > \tau_{\text{repair}} \ \vee \ D_t > \tau_{\text{disagree}}\}$
if $z_t = 1$ **then**
  $E_t^{\text{new}} \leftarrow E_{\text{new}}(h_t, y_t), \quad \mathcal{S}_t^{\text{new}} \leftarrow \text{AddConcept}(\mathcal{S}_t, h_t, y_t, E_t^{\text{new}}; \mathcal{B}_t^{\text{prop}})$
  $\widehat{\Delta}_t^{\text{refine}} \leftarrow \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_t^{\text{new}}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} |E_t^{\text{new}}|$
  $r_t \leftarrow \mathbf{1}\{\widehat{\Delta}_t^{\text{refine}} > \max\{\widehat{\Delta}_t^{\text{repair}}, 0\} + \tau + 4\epsilon_m \ \wedge \ D_t > \tau_{\text{disagree}}\}$
  if $r_t = 1$ **then**
    $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t^{\text{new}}$
    return $\mathcal{S}_{t+1}$
  end if
end if
$\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t^{\text{repair}}$
return $\mathcal{S}_{t+1}$

Definition J.10 (Guarded monotone local repair)**.** *A guarded repair operator is monotone on scoring buffer $\mathcal{B}_t^{\text{score}}$ if it commits a candidate state $\mathcal{S}_{t+1}$ only when local memories and retrieval centroids may change, anchors and edges remain fixed, and*

$$\widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) \leq \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}).$$

The identity candidate $\mathcal{S}_{t+1} = \mathcal{S}_t$ is always allowed, so the guard is feasible even when every nontrivial repair would hurt the scoring buffer.

Theorem J.11 (Refine-or-repair objective descent)**.** *Let the local scoring objective on a fixed scoring buffer $\mathcal{B}_t^{\text{score}}$ be*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}}) + \lambda_{\text{concept}} N_{\text{concept}}(\mathcal{S}) + \lambda_{\text{edge}} \left|E(\mathcal{S})\right|.$$

If repair is committed by the guarded rule (12), then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement adding one anchored concept and actual edge set E_{new} is accepted only under (15), then the empirical penalized objective $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\widehat{\Delta}_{\text{repair}} + \tau + 4\epsilon_m$ and in particular by more than $\tau + 4\epsilon_m$.

This is a local-buffer statement. It does not imply monotone descent of a global objective when the scoring buffer changes over time; a global finiteness claim requires a common potential or validation objective.

Theorem J.12 (Statistically safe adaptive refinement)**.** *Assume losses lie in $[0, 1]$. Let $(\mathfrak{H}_t)$ be the filtration containing the past stream, the current CGA state, and all proposal-buffer information available before scoring at time t. Let $\mathcal{C}_t$ be a finite candidate class that is predictable with respect to $\mathfrak{H}_t$, and let $\mathcal{B}_t^{\text{score}}$ contain m_t fresh held-out samples, or samples not previously used to choose proposals or update the state, independent of $\mathfrak{H}_t$ conditional on the current scoring distribution. For candidate g, let $\widehat{G}_t(g)$ be the empirical raw risk decrease on $\mathcal{B}_t^{\text{score}}$ before complexity cost, and let $G_t(g)$ be the corresponding population raw gain. Define*

$$\epsilon_t = \sqrt{\frac{\log(4t^2(|\mathcal{C}_t| + 1)/\delta)}{2m_t}}.$$

Then, with probability at least $1 - \delta$, for every time t and every $g \in \mathcal{C}_t$,

$$\left|\widehat{G}_t(g) - G_t(g)\right| \leq 2\epsilon_t.$$

Hence an accepted refinement satisfying

$$\widehat{G}_t(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_t$$

equivalently $\widehat{\Delta}_t^{\text{pen}}(g) = \widehat{G}_t(g) - \lambda \operatorname{cost}(g) > \tau + 2\epsilon_t$, has positive population penalized gain after paying complexity cost and margin τ, uniformly over all accepted times.

Corollary J.13 (Bounded language-model refinement score)**.** *For language modeling, raw next-token negative log likelihood is unbounded. Fix $L_{\max} > 0$ and define the clipped normalized score*

$$\ell_{\text{clip}}(x_{\leq t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\leq t}), L_{\max}\}.$$

If refinement proposal and scoring use ℓ_{clip}, then Theorem J.12 applies to clipped-loss refinement gain. An accepted refinement has positive clipped population gain after cost and margin with the stated high probability. This corollary does not claim positive gain for the raw unbounded negative log likelihood without additional tail assumptions.

Theorem J.14 (Anchor-stable CGA information structure)**.** *Let*

$$\mathcal{F}_t^{\text{CGA}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h - a_i\|_2^2 \leq \|h - a_j\|_2^2\} : 1 \leq i, j \leq N_t\Big).$$

If a repair step changes only retrieval centroids c_j, local memories M_j, and counts n_j, while keeping anchors a_j and edges E_t fixed, then $\mathcal{F}_{t+1}^{\text{CGA}} = \mathcal{F}_t^{\text{CGA}}$. If a refinement adds a new anchor and new edge generators, then the updated information structure is the join of $\mathcal{F}_t^{\text{CGA}}$ with the new anchor-order events and the new edge generators.

Thus the edge skeleton realizes the minimal join operation from Section G. The exact hard-route CGA adds anchor-order events as routing infrastructure whenever a new anchor is inserted. Consequently, Hard-CGA is a structured implementation of refinement, but its exact-route information update is not always the single-event minimal refinement in the strict Section G sense.

Corollary J.15 (Finite number of accepted refinements)**.** *If there is a common nonnegative potential $\Phi(\mathcal{S})$ and every accepted refinement decreases Φ by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$. In particular, this applies to a fixed scoring buffer or validation objective. The local-buffer descent theorem alone should not be read as a global finiteness result when the scoring buffer changes over time.*

J.5 Data-structure operations

Definition J.16 (CGA data-structure operations)**.** *Viewed as a dynamic partition data structure, CGA implements four operations:*

$$\mathrm{Query}(h), \qquad \mathrm{Repair}(h, y), \qquad \mathrm{Refine}(h, y, g), \qquad \mathrm{Score}(g, \mathcal{B}).$$

The maintained edge-skeleton invariant is

$$\begin{aligned} \mathrm{Repair} \quad &\Longrightarrow \quad \mathcal{P}^G(\mathcal{S}_{t+1}) = \mathcal{P}^G(\mathcal{S}_t), \\ \mathrm{Refine}(g) \quad &\Longrightarrow \quad \mathcal{P}^G(\mathcal{S}_{t+1}) = \mathcal{P}^G(\mathcal{S}_t) \vee \sigma(g), \end{aligned}$$

where $\mathcal{P}^G(\mathcal{S}_t)$ is the edge-generated skeleton partition. The exact hard-route invariant is

$$\begin{aligned} \mathrm{Repair} \quad &\Longrightarrow \quad \mathcal{F}^{\mathrm{CGA}}(\mathcal{S}_{t+1}) = \mathcal{F}^{\mathrm{CGA}}(\mathcal{S}_t), \\ \mathrm{Refine} \quad &\Longrightarrow \quad \mathcal{F}^{\mathrm{CGA}}(\mathcal{S}_{t+1}) = \mathcal{F}^{\mathrm{CGA}}(\mathcal{S}_t) \vee \sigma(g_e : e \in E_{\mathrm{new}}) \vee \sigma(\textit{new anchor-order events}). \end{aligned}$$

Thus repair modifies payloads on the current structure, while refinement changes the structure by adding both edge generators and the order information induced by new anchors.

Theorem J.17 (CGA data-structure cost)**.** *Assume N_t concepts in $\mathbb{R}^d$, candidate set size $|N(h)| = k$, selected incident edge count $Q(h)$, retrieval cost $C_{\mathrm{ret}}(N_t, d)$, and selected local-memory evaluation cost C_{mem}. Then*

$$\mathrm{Query}(h) = O(C_{\mathrm{ret}}(N_t, d) + Q(h)d + kd + C_{\mathrm{mem}}).$$

Here $Q(h)$ is an edge count and each dense linear edge gate costs $O(d)$ to evaluate. A repair adds the cost of scoring the proposed local-memory candidates on the scoring buffer. A refinement creating one anchor-centroid pair and q guarded linear distinction edges adds $O(d + qd)$ state-update time, plus the indexing or comparison bookkeeping used to maintain anchor-order queries. With sparse categorical memories of average support $\bar{s}$, the state uses

$$O(N_t(2d + \bar{s}) + |E_t|\, d)$$

memory, up to indexing overhead.

The theorem is an implementation bound rather than a statistical guarantee. It says that CGA realizes the abstract operation $\Pi \mapsto \Pi \vee U$ on the edge skeleton and the corresponding join update on $\mathcal{F}^{\mathrm{CGA}}$ with local graph updates and sparse routing, provided the candidate retrieval layer supplies the relevant hard-route concepts.

Comparison to gradient and memory baselines. Backpropagation is not itself a repair or refinement operation; it is an optimizer. If it updates only parameters downstream of a fixed information map, it is a fixed-information repair update in the sense of Definition E.3. If it updates the encoder or routing representation, it may change the induced partition and is outside the repair-only lower bound. Table 5 summarizes the resulting computational comparison against common online optimizers, caches, trees, and parameter-efficient adaptation modules [Rumelhart et al., 1986, Kingma and Ba, 2015, Grave et al., 2017, Khandelwal et al., 2020, Domingos and Hulten, 2000, Houlsby et al., 2019, Hu et al., 2022]. Let P_{tr} be the number of trainable parameters updated by backpropagation, A_{bp} the activation storage required for the backward pass over the adaptation window, M the number of stored examples in a cache or kNN memory, D_{tree} the tree depth, S_{leaf} the per-leaf sufficient statistics, and $\bar{s}$ the average sparse categorical support per CGA concept. The table lists runtime-adaptation overhead beyond any shared frozen forward pass that computes h.

Method	Information operation	Per-round adaptation / read overhead	Runtime state memory
Full backprop + Adam	May change information map if encoder/routing parameters move	Backward/update through P_{tr} parameters; activation storage for the adaptation window	Weights, gradients, and Adam moments: $O(P_{\mathrm{tr}})$ each, plus $O(A_{\mathrm{bp}})$ activations
Head-only or linear-probe backprop	Fixed-information repair when features $h = I(\omega)$ are frozen	Backward/update through head parameters P_{head}	$O(P_{\mathrm{head}})$ weights plus optimizer state and head activations
Adapters / LoRA online	Repair if downstream of fixed I; otherwise may change the information map	Backward/update through adapter parameters P_{ad} in selected layers	$O(P_{\mathrm{ad}})$ weights plus optimizer state and selected-layer activations
Cache or budgeted kNN	Stores examples under a fixed representation; no explicit refinement test	Retrieval $C_{\mathrm{ret}}(M, d)$ plus aggregation over k neighbors	$O(M(d + \bar{s}))$ keys and values, plus index overhead
Online tree / Hoeffding tree	Refines a rooted leaf partition	Query $O(D_{\mathrm{tree}})$; update split statistics on visited leaf	$O(\#\mathrm{leaves} \cdot S_{\mathrm{leaf}})$
CGA, exact Hard-CGA	Refines graph-generated partition by adding binary generators	Query $O(N_t d + Q(h)d + kd + C_{\mathrm{mem}})$ with exact retrieval; accepted refinement adds $O(d + qd)$	$O(N_t(2d + \bar{s}) + \lvert E_t \rvert\, d)$
CGA, sparse Soft-CGA	Approximate Hard-CGA under candidate recall and margins	Query $O(C_{\mathrm{ret}}(N_t, d) + Q(h)d + kd + C_{\mathrm{mem}})$; accepted refinement adds $O(d + qd)$	$O(N_t(2d + \bar{s}) + \lvert E_t \rvert\, d)$ plus index overhead

Table 5: Runtime-adaptation cost comparison. CGA does not claim to dominate unconstrained full backpropagation statistically. The theoretical separation applies to fixed-information repair methods: when the representation quotient is fixed and has positive refinement gap, they incur the lower bound in Theorem F.3, while CGA can pay local graph-memory cost to add a distinction. Full backpropagation through the representation can also change the quotient, but pays backward-pass, activation, and optimizer-state costs.

J.6 Canonical local memory

Definition J.18 (Calibrated categorical local memory)**.** *The local memory M_j is the repair mechanism inside an already chosen concept. For classification and language modeling, the main-text canonical memory is a calibrated categorical memory rather than an unconstrained embedding score, consistent with proper probabilistic scores and calibration diagnostics for probability forecasts [Brier, 1950, Gneiting and Raftery, 2007, Guo et al., 2017]. Concept j stores token or label counts $n_{j,y}$ and $n_j = \sum_y n_{j,y}$. Given a backbone distribution $p_0(y \mid h)$ and smoothing strength $\alpha_0 > 0$, define*

$$p_j(y \mid h) = \frac{n_{j,y} + \alpha_0 p_0(y \mid h)}{n_j + \alpha_0}. \tag{16}$$

The routed memory distribution is

$$p_{\mathrm{mem}}(y \mid h) = \sum_{j \in N(h)} \alpha_j(h) p_j(y \mid h), \tag{17}$$

and the final prediction is the convex mixture

$$p_{\mathrm{CGA}}(y \mid h) = (1 - \lambda_t) p_0(y \mid h) + \lambda_t p_{\mathrm{mem}}(y \mid h), \qquad \lambda_t \in [0, 1]. \tag{18}$$

This keeps the read path a valid probability distribution. It also gives the simple pointwise safety bound

$$-\log p_{\mathrm{CGA}}(y \mid h) \le -\log p_0(y \mid h) - \log(1 - \lambda_t)$$

whenever $\lambda_t < 1$. For log-loss experiments we use a stronger cumulative safeguard based on standard prediction-with-expert-advice and aggregating-algorithm bounds [Vovk, 1990, Freund and Schapire, 1997, Cesa-Bianchi and Lugosi, 2006]. Let $\Lambda = \{\lambda_1, \dots, \lambda_K\} \subseteq [0, 1]$ with $0 \in \Lambda$, define

$$q_{t,k}(y) = (1 - \lambda_k)p_0(y \mid h_t) + \lambda_k p_{\mathrm{mem}}(y \mid h_t),$$

and predict with a mixture $p_t(y) = \sum_k w_{t,k} q_{t,k}(y)$. After observing y_t, update the trust weights by

$$w_{t+1,k} = \frac{w_{t,k} q_{t,k}(y_t)}{\sum_{\ell=1}^{K} w_{t,\ell} q_{t,\ell}(y_t)}. \tag{19}$$

Theorem J.19 (No-harm log-loss trust aggregation, standard)**.** *Initialize $w_{1,k} = 1/K$ and update by (19). For any outcome sequence,*

$$\sum_{t=1}^{T} -\log p_t(y_t) \le \min_{k \in [K]} \sum_{t=1}^{T} -\log q_{t,k}(y_t) + \log K.$$

Since $0 \in \Lambda$, one expert is the frozen backbone predictor, so

$$\sum_{t=1}^{T} -\log p_t(y_t) \le \sum_{t=1}^{T} -\log p_0(y_t \mid h_t) + \log K.$$

Proof. Let $Z_t = \sum_k w_{1,k} \prod_{s=1}^{t} q_{s,k}(y_s)$. The posterior update implies $p_t(y_t) = Z_t/Z_{t-1}$, hence $\prod_{t=1}^{T} p_t(y_t) = Z_T$. For every k, $Z_T \ge K^{-1} \prod_{t=1}^{T} q_{t,k}(y_t)$. Taking negative logarithms and minimizing over k gives the first bound. The second follows from the $\lambda = 0$ expert. □

Because $p_0(y \mid h)$, $M_j(h)$, and the soft weights can depend on the hidden state, the practical predictor need not be constant on atoms of the edge partition alone. The formal interpretation is the base-information convention from Section D: let

$$\mathcal{B} = \sigma(p_0(\cdot \mid h), \text{fixed pre-outcome features used outside CGA routing}).$$

Then CGA refinements are claims about the additional runtime information in $\mathcal{F}_t^{\mathrm{CGA}}$, and predictors are measured as $\mathcal{B} \vee \mathcal{F}_t^{\mathrm{CGA}}$-measurable. The hidden state may be used by the algorithm to compute admissible distinctions, proposal scores, and restricted routing functions, but it is not treated as arbitrary freely conditionable information in the unrestricted Bayes-risk theory. If full h is admitted into $\mathcal{B}$, then all edge predicates and anchor-order events are already $\mathcal{B}$-measurable and the partition-level refinement gap collapses to zero. In the hard one-hot special case, where each memory returns an atomwise constant action and no extra base features are used, this reduces to strict partition measurability; h-dependent soft memories should be read as restricted-class or approximation statements.

Repair is the sufficient-statistic update. For every selected concept $j \in N(h)$,

$$n_{j,y}^{+} = n_{j,y} + \alpha_j(h), \qquad n_j^{+} = n_j + \alpha_j(h),$$

with all unselected counts unchanged. The low-cost hard-update variant is recovered by setting $\alpha_{j^\star} = 1$ for the highest-weight concept and $\alpha_j = 0$ for all $j \neq j^\star$. Embedding-mean memories and ridge or projection-style local repair are useful implementation variants, but they should be evaluated with calibration metrics before being used for log-loss claims.

J.7 From CGA to CGA-LM

The generic CGA object above can be attached to many local memories. For sequence models, Appendix K gives CGA-LM as an attention-replacement architecture with a causal read-before-write protocol and detached runtime graph updates.

K CGA-LM: Replacing Token Attention with Concept Graph Attention

For language modeling, the world state is a prefix and the consequence is the next token. CGA-LM replaces the causal attention sublayer inside a decoder block with Concept Graph Attention:

$$\mathrm{CGABlock}(X) = X + \mathrm{CGA}(\mathrm{Norm}(X)).$$

The block has the same external shape contract as attention, $[B, L, d_{\mathrm{model}}] \mapsto [B, L, d_{\mathrm{model}}]$. The residual stream, layer normalization, embeddings, and language-model head are otherwise standard decoder components. In a Transformer-style decoder, CGA can be paired with the usual MLP sublayer. For the state-matched PoST LM runs, the default implementation uses the same mixer-only residual scaffold as the Mamba-2 baseline; an MLP sublayer is an explicit configuration option rather than part of the main comparison.

K.1 Fixed concept memory

At layer ℓ, CGA-LM keeps a fixed bank of C concept identities $a_{\ell hj} \in \mathbb{R}^{d_a}$ for each head h and concept slot j. These identities are fixed addresses shared across sequences. The online state is the value memory

$$M_t^\ell \in \mathbb{R}^{H \times C \times d_v^c}.$$

Each token produces read queries, write keys, write values, write gates, and retention-decay projections:

$$q_t = Q_\ell x_t, \qquad k_t = K_\ell x_t, \qquad v_t = V_\ell x_t, \qquad g_t = \sigma(w_g^\top x_t + b_g).$$

The read is causal: token t reads only concept values produced by earlier writes. For concept slots $j = 1, \ldots, C$,

$$\alpha_{thj} = \mathrm{softmax}_j \left(\gamma_\ell q_{th}^\top a_{\ell hj}\right), \qquad z_{th} = \sum_{j=1}^{C} \alpha_{thj} M_{t,hj}^\ell, \qquad y_t = O_\ell[z_{t1}; \ldots; z_{tH}].$$

Multi-head CGA uses independent address and value subspaces per head and concatenates the head outputs before the output projection, exactly as multi-head attention does.

K.2 Read before write

The update order is the same during training and autoregressive inference:

$$\text{read from } M_t^\ell \to \text{emit } y_t \to \text{decay and write } (k_t, v_t, g_t) \to \text{update } M_{t+1}^\ell.$$

This prevents a token from reading its own write. The inference-time update is the test-time learning mechanism in CGA-LM: the model weights are fixed, but the concept value memory changes after the causal write and future tokens condition on that updated memory.

For a delayed write key $\bar{k}_t = k_{t-\delta}$, CGA computes write weights

$$\beta_{thj} = \mathrm{softmax}_j \left(\gamma_\ell \bar{k}_{th}^\top a_{\ell hj}\right).$$

With retention factor ρ_{th}, the token-level streaming update is

$$M_{t+1,hj}^\ell = \rho_{th} M_{t,hj}^\ell + g_t \beta_{thj} v_{th}.$$

For parallel training, the implementation groups tokens into causal blocks: all tokens in a block read the memory from before the block, then their writes are accumulated into one block update. Setting block size one recovers the exact token-streaming update.

Both MQAR and language-model variants use a learned write gate. The current experiments also use a trainable retention factor with a PoST-style ordered decay parameterization. This is a parameterization choice for the forgetting spectrum, not the definition of CGA itself; CGA can be run with a free retention parameterization for ablations.

K.3 Training boundary

Slow neural parameters are trained by ordinary next-token backpropagation through both the CGA read and the continuous write. Gradients flow through the read weights, write weights, value projection, write gate, retention factors, and output projection. At inference time there is no gradient step and no update to model weights. The only state that changes is the forward-updated concept value memory. Thus CGA-LM is not a backprop-free language model and not a test-time optimizer over hidden model weights. It is an attention replacement whose fast state is a bounded concept memory.

Algorithm 4 CGALAYERFORWARD

```
Input: hidden states x_{1:L}, fixed concept identities a_{1:C}, zero value memory M_0
for causal blocks do
    project block queries, write keys, values, gates, and retention rates
    read fixed concept memory with continuous attention weights
    emit block outputs through the output projection
    form delayed write keys and write weights
    decay old value memory and add gated continuous writes
end for
return contextualized hidden states
```

K.4 Scaling path

The reference CGA implementation uses dense concept reads. This is appropriate for state-equalized MQAR and for the first language-model runs with $C = 128$. The large-scale implementation target is a fused dense CGA kernel that combines concept-score computation, softmax, value read, retention, and gated write accumulation inside the mixer. For substantially larger concept banks, sparse concept retrieval and sharded concept banks become systems work. The paper therefore separates the architectural claim from the kernel claim: CGA changes the memory basis of attention from tokens to concepts; optimized implementations should then choose dense or sparse concept attention depending on the concept budget.

L Language-Model Evaluation Details

This appendix specifies the large-scale CGA-LM experiment in Section 6.2. The goal is to match the PoST language-model evaluation protocol while changing only the sequence mixer. All models are pretrained as base language models and evaluated without instruction tuning or in-context examples.

L.1 Training protocol

Table 6: CGA-LM training protocol for the large language-model evaluation.

Parameter	Value
Training data	FineWeb-Edu [Lozhkov et al., 2024]
Training context length T_{train}	2,048
Tokenizer	Llama-3.1 tokenizer [Dubey et al., 2024]
Hardware	8× H200
Optimizer	AdamW [Loshchilov and Hutter, 2019]
Warmup	1% of total steps
Precision	BF16 mixed precision
Gradient clipping	$\|\nabla\|_2 \leq 1.0$
Training tokens (180M / 440M)	4B / 9B
Learning rate (180M / 440M)	6×10^{-4} / 3×10^{-4}, cosine to 10^{-5}

L.2 State-matched CGA configuration

The CGA state is matched to the recurrent state size used by the PoST Mamba-2 configurations. For Mamba-2, the per-layer runtime state is $d_{\text{inner}} d_{\text{state}}$. For CGA-LM, the online state is CHd_v^c. We therefore use $C = 128$ and $d_v^c = 64$ at both scales:

Table 7: State-matched CGA-LM configurations. Concept identities are fixed layer parameters; the online state is only the concept value memory.

Parameter	**180M**	**440M**
d_{model}	768	1,024
Number of layers	24	48
Heads H	24	32
Concepts C	128	128
Concept value width d_v^c	64	64
CGA online state CHd_v^c	196,608	262,144
Matched Mamba-2 state $d_{\text{inner}} d_{\text{state}}$	1,536×128	2,048×128
Concept identities	fixed random unit addresses	fixed random unit addresses
Read/write addressing	untied projections	untied projections
Retention	trainable, PoST-style parameterization	trainable, PoST-style parameterization

L.3 Evaluation protocol

We use the Language Model Evaluation Harness [Gao et al., 2024] in a zero-shot setting. The reported tasks are LAMBADA, HellaSwag, PIQA, ARC-Easy, ARC-Challenge, WinoGrande, and OpenBookQA, matching Table 2. For reproducibility and systems comparison, each checkpoint should report: validation loss, validation perplexity, zero-shot task metrics, training tokens per second, prefill latency, decode latency, parameter count, online state size, and CGA diagnostics for write gate, read scale, retention, and decay. The latency table is generated by `scripts/benchmark_lm_latency.py`, which uses synthetic tokens so that all checkpoints are measured with identical batch size, context length, dtype, and hardware.

M From Distinctions to Projection Memory

CGA uses refinement and repair in the same runtime object. Distinction learning asks what the learner can tell apart. Projection learning assumes such a representation and asks how a state inside it should be minimally repaired. The repair side is related to Bregman projection, Moore-Penrose projection, ridge/Tikhonov repair, passive-aggressive learning, online regression, and fast-weight memory updates [Bregman, 1967, Penrose, 1955, Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970, Crammer et al., 2006, Vovk, 1997, Ba et al., 2016, Schlag et al., 2021]. This section makes the relation formal in the finite one-hot case.

M.1 One-hot partition memory

Let $\mathcal{P} = \{B_1, \dots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}} : \Omega \to \mathbb{R}^n$ be its one-hot feature map:

$$\phi_{\mathcal{P}}(\omega) = e_i \qquad \text{if} \qquad \omega \in B_i.$$

A memory matrix $W \in \mathbb{R}^{m \times n}$ defines a value map

$$f_W(\omega) = W\phi_{\mathcal{P}}(\omega).$$

This is exactly a table over the atoms of $\mathcal{P}$: every state inside the same atom receives the same value.

Theorem M.1 (One-hot partition memory)**.** *For every $W \in \mathbb{R}^{m \times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is represented by some W through $f = f_W$.*

Thus a one-hot associative memory is not merely analogous to a partition. It is an exact finite representation of all vector-valued predictors measurable with respect to that partition.

M.2 CPM conflict as missing distinction

We take K to be the matrix whose columns are stored keys, so $KK^\dagger$ is the orthogonal projector onto their span. For old key matrices K, a new key k has CPM innovation

$$z = (I - KK^\dagger)k. \tag{20}$$

Conservative Projection Memory detects conflict when a new key has zero innovation but nonzero residual. In the one-hot finite representation, this diagnostic is exactly the statement that two world states have been placed in the same atom despite requiring different values.

Theorem M.2 (CPM conflict equals missing distinction)**.** *Let $\omega, \omega' \in B_i$ for an atom $B_i \in \mathcal{P}$, and suppose required values satisfy $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If $We_i = v(\omega)$ has already been committed, then evidence requiring $We_i = v(\omega')$ has CPM innovation $z = 0$ and residual $e \neq 0$.*

The theorem explains why a projection update should stop. The key is not new; the old information structure maps both states to the same coordinate. The residual is nonzero because the consequence differs. The missing update is a refinement of $\mathcal{P}$ that separates ω from ω'.

M.3 The hierarchy

The resulting hierarchy is

distinction refinement $\supset$ projection repair $\supset$ implicit gradient $\supset$ explicit gradient descent.

Distinction refinement changes the sigma-algebra $\mathcal{F}_t$. Projection repair changes a state s_t inside a fixed information geometry. Gradient descent is the linearized ℓ_2 special case of soft projection, and backpropagation is the mechanism that computes such gradients in differentiable static graphs [Rumelhart et al., 1986].

This is not a rejection of gradient learning or projection memory. It is a separation of roles. When the representation is adequate, repair and optimization are the right tools. When the representation identifies states that consequences require it to separate, the first learning act is to create the missing distinction.

N CGA Implementation Extensions

This appendix collects CGA mechanics and large-model details that are useful for implementations but secondary to the main refine-or-repair argument.

N.1 Trees as a hard-routing special case

A tree stores distinctions as a hierarchy. A mistake near the root changes every downstream routing decision. A graph stores distinctions locally between concepts. Creating a new concept does not rewrite old paths; it adds a local alternative and edges explaining why the new concept differs from its neighbors. Trees are still contained in the view below, but as a constrained hard-routing case.

Proposition N.1 (Trees as generalized hard-CGAs)**.** *Given a binary distinction tree, construct one generalized hard-route CGA concept for each leaf and attach the path distinctions as hard eligibility constraints for that concept. If the generalized hard route selects the unique eligible leaf concept and each concept memory equals the tree leaf predictor, then the generalized CGA and the tree make identical predictions.*

Thus the distinction-graph view contains trees after allowing arbitrary concept-level eligibility constraints. A decision tree is the sparse case in which distinctions are forced into a rooted hierarchy and routing is hard and single-path.

N.2 CGA data-structure cost

Let

$$Q(h) = \sum_{j \in N(h)} |\mathrm{Inc}_t(j)|$$

be the number of incident edge gates evaluated for the selected candidates. With exact nearest-neighbor search, prediction costs

$$O(N_t d + Q(h)d + |N(h)|\, d + C_{\text{local}}),$$

where C_{local} is the cost of evaluating the selected local memories and the $Q(h)d$ term evaluates dense linear edge gates. With approximate nearest-neighbor indexing, the $N_t d$ term can be replaced by the query cost of the index [Indyk and Motwani, 1998, Johnson et al., 2019]. Creating an anchor-centroid pair costs $O(d)$ plus the cost of initializing its memory and the bookkeeping used for anchor-order queries. Adding q guarded distinction edges costs $O(qd)$. If each concept stores a sparse categorical memory with average support size $\bar{s}$, runtime memory is

$$O(N_t(2d + \bar{s}) + |E_t|\, d).$$

An embedding-mean variant instead uses $O(N_t(2d + d_e) + |E_t|\, d)$ memory, but it is not a calibrated log-loss memory without an additional probability model.

Theorem N.2 (CGA data-structure bound)**.** *For exact search over N_t centroids in $\mathbb{R}^d$, one edge-gated CGA prediction costs $O(N_t d + Qd + |N|\, d + C_{\text{local}})$, where $N = N(h)$ and $Q = Q(h)$ is an edge count. One observation that repairs a selected concept costs this plus the guarded local repair scoring cost. One observation that creates a new anchor-centroid pair and q guarded distinction edges costs an additional $O(d + qd)$ plus anchor-order query bookkeeping. With sparse categorical memories of average support $\bar{s}$, the runtime state uses $O(N_t(2d + \bar{s}) + |E_t|\, d)$ memory.*

N.3 Ridge local memory

For few-shot adaptation or local readout repair, a memory stores support embeddings $X_{\text{sup}} \in \mathbb{R}^{n \times d}$ and labels $Y \in \mathbb{R}^{n \times C}$. The closed-form adapter is the dual ridge/Tikhonov solution [Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970]:

$$\Gamma = (X_{\text{sup}} X_{\text{sup}}^\top + \lambda I)^{-1} Y, \qquad \text{logits}(z) = z X_{\text{sup}}^\top \Gamma.$$

Projection Memory and CPM are advanced repair rules inside M_j after the graph has fixed the relevant local concept; Appendix M treats that relation directly.

N.4 Training modes for representation drift

Anchors, centroids, and edges live in query space. If the encoder or query map changes rapidly while the CGA state persists, old anchors and edges describe stale geometry. Reports should therefore state which training mode is used.

Mode A: frozen encoder. Train the backbone and query map first, then freeze ϕ_θ, Q, and the backbone distribution used for CGA smoothing while running CGA runtime learning. This is the cleanest protocol for the first experiments and is the default interpretation of the theory.

Mode B: EMA query encoder. Maintain a slowly moving query encoder

$$\bar{\theta} \leftarrow \beta \bar{\theta} + (1 - \beta)\theta,$$

and create anchors, centroids, and edge generators in the query space of $\phi_{\bar{\theta}}$ and $Q_{\bar{\theta}}$. The backbone can still train, but the CGA state is tied to the slower geometry.

Mode C: periodic state rebuild. During end-to-end training, rebuild or clear the CGA state every K optimization steps, or rebuild it from a replay buffer under the current query map. The formal guarantees then apply within a rebuild interval. Cross-interval state carryover is a systems approximation, not a claim of anchor-stable refinement.

N.5 Non-canonical adapter extension

Beyond the categorical next-token memory in Appendix K, a richer optional adapter-style layer stores a local linear repair map, related to parameter-efficient adapter and LoRA modules but updated as a

local CGA memory [Houlsby et al., 2019, Hu et al., 2022]. It requires an explicit residual target and should not be conflated with the calibrated next-token memory mechanism:

$$\mathcal{S}_t^{\mathrm{adp}} = \{a_j, c_j, W_j, n_j\}_{j=1}^{N_t} \cup G_t, \qquad W_j \in \mathbb{R}^{d\times r}.$$

Let $K : \mathbb{R}^d \to \mathbb{R}^r$ be a learned or fixed key projection and set $k_t = Kh_t$. The adapter read is

$$\Delta h_t = \sum_{j\in N(q_t)} \alpha_j(q_t) W_j k_t, \qquad \tilde{h}_t = h_t + \gamma_\theta(h_t)\Delta h_t.$$

Here $W_j k_t$ is a hidden-space residual, so the write rule requires an explicit residual target $v_t \in \mathbb{R}^d$. Let $P_{\mathrm{emb}\to h} : \mathbb{R}^{d_e} \to \mathbb{R}^d$ be a chosen projection from output-embedding space to hidden-residual space. Natural choices include a token-embedding target $P_{\mathrm{emb}\to h} e_{x_{t+1}}$, a teacher-forced output-error target $P_{\mathrm{emb}\to h}(e_{x_{t+1}} - \hat{e}_t)$, or a distillation target $h_t^{\mathrm{teacher}} - h_t$. Without such a target, the adapter is only an architectural extension.

Given a local residual target, the projected rank-one repair for a selected concept is the same residual-correction pattern that appears in passive-aggressive updates, RLS/ridge-style online least squares, and delta-rule fast-weight memories [Crammer et al., 2006, Vovk, 1997, Schlag et al., 2021]:

$$W_j^+ = W_j + \frac{\eta\alpha_j(q_t)}{1 + \eta\alpha_j(q_t)\,\|k_t\|_2^2}\big(v_t - W_j k_t\big)k_t^\top.$$

Equivalently, a memory may maintain small ridge sufficient statistics and solve the local adapter in closed form [Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970].

N.6 Cost and systems notes

During sequential detached training, backpropagation sees the selected read computation, while the CGA state is updated in the same order as streaming inference. The important comparison is with test-time gradient adaptation [Sun et al., 2020, Wang et al., 2021, Niu et al., 2022, Zhang et al., 2022, Sun et al., 2025]. A TTT-style layer typically performs an inner backward pass at inference time. A CGA layer performs retrieval, sparse local read, and closed-form or sufficient-statistic repair. The cost is forward-only:

$$\text{CGA inference} = \text{backbone forward} + \text{retrieval} + \text{sparse local repair}.$$

Theorem N.3 (CGA layer cost)**.** *Consider a CGA adapter extension with hidden dimension d, query dimension d_q, local rank r, candidate count $|N|$, selected incident edge-gate count Q_N, and concept-bank query cost $C_{\mathrm{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\mathrm{ret}}(N_t, d_q) + Q_N d_q + |N|\,dr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\mathrm{ret}}(N_t, d_q) + Q_N d_q + |N|\,(d_q + dr)).$$

For a block of length L, the CGA state history need not be stored for backpropagation; the additional activation memory is the selected read computation, not the full sequence of memory states. The CGA-LM mixer replaces the $|N|\,dr$ adapter term by sparse concept lookup, edge-gated value aggregation, and detached graph-write costs.

The concept bank can be sharded by the same principle used in retrieval and sparse expert systems. Each device owns a subset of concepts. A query is routed to a small set of shards, each shard returns local top-k candidates, and a final top-k merge selects the memories used by the read path. Only selected memories receive gradients. Detached writes are sent back to the owning shards.

Budgeting is necessary for long training. A practical CGA layer maintains $N_t \le N_{\max}$, merges only compatible unprotected concepts, and otherwise evicts or compresses low-score memories. These budget actions are maintenance operations, not learning operations in the formal refine-or-repair sense. The refine-or-repair theorems apply between maintenance events; merge, eviction, and compression may destroy information and must be evaluated as resource-management approximations.

N.7 Closed-form local repair

Ridge memory is the main closed-form adapter used in our checks. Let $X_{\sup} \in \mathbb{R}^{n\times d}$ be normalized support embeddings stored by a CGA memory and let $Y \in \mathbb{R}^{n\times C}$ be one-hot consequences. The adapter computes the dual ridge/Tikhonov solution [Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970]

$$\Gamma = (X_{\sup}X_{\sup}^{\top} + \lambda I)^{-1}Y, \qquad \mathrm{logits}(z) = zX_{\sup}^{\top}\Gamma.$$

This is projection-style repair: it fits the local adapter in one linear solve rather than by iterated backpropagation. In a large model, the same idea can be applied per concept, per document, or per retrieved shard after the relevant distinction structure has been fixed.

O Minimal Refinement Proofs

The statements in this appendix use standard facts about finite partitions, generated sigma-algebras, and measurability [Billingsley, 1995, Kallenberg, 2002]. They are recalled only to fix notation for the runtime-learning interpretation: refinement is the minimal information update needed by new evidence.

Theorem O.1 (Minimal finite refinement, formal version of Theorem G.2)**.** *Let Ω be finite, let $\mathcal{P}$ be a partition of Ω, and let $D \subseteq \Omega$. Define*

$$\mathcal{P}_D = \{B \cap D : B \in \mathcal{P}, B \cap D \neq \varnothing\} \cup \{B \cap D^c : B \in \mathcal{P}, B \cap D^c \neq \varnothing\}.$$

Then:

- **Part 1.** *$\mathcal{P}_D$ is a partition of Ω.*
- **Part 2.** *$\mathcal{P}_D$ refines $\mathcal{P}$.*
- **Part 3.** *D is $\mathcal{P}_D$-measurable.*
- **Part 4.** *If $\mathcal{Q}$ refines $\mathcal{P}$ and D is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_D$.*

Consequently, $\mathcal{P}_D$ is the unique minimal refinement of $\mathcal{P}$ making D measurable.

Proof. For each $B \in \mathcal{P}$, the nonempty sets among $B \cap D$ and $B \cap D^c$ are disjoint and have union B. Since the atoms of $\mathcal{P}$ are disjoint and cover Ω, the atoms of $\mathcal{P}_D$ are disjoint and cover Ω. Thus Part 1 holds.

Every atom of $\mathcal{P}_D$ is contained in some $B \in \mathcal{P}$, so Part 2 holds. Also

$$D = \bigcup_{B\in\mathcal{P}} (B \cap D),$$

where empty intersections are ignored. Hence D is a union of atoms of $\mathcal{P}_D$, proving Part 3.

Let $\mathcal{Q}$ refine $\mathcal{P}$ and make D measurable. For any atom $Q \in \mathcal{Q}$, there exists $B \in \mathcal{P}$ with

$$Q \subseteq B.$$

Since D is $\mathcal{Q}$-measurable, each atom of $\mathcal{Q}$ is either contained in D or disjoint from D. Hence

$$Q \subseteq B \cap D \qquad \text{or} \qquad Q \subseteq B \cap D^c.$$

Thus each atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}_D$, so $\mathcal{Q}$ refines $\mathcal{P}_D$. This proves Part 4.

If another partition has the same minimal property, it refines $\mathcal{P}_D$ and is refined by $\mathcal{P}_D$, so the two partitions have the same atoms. □

Theorem O.2 (Minimal sigma-algebra refinement, formal version of Theorem G.4)**.** *Let $(\Omega, \mathcal{E})$ be a measurable space, let $\mathcal{F} \subseteq \mathcal{E}$ be a sigma-algebra, and let $U : \Omega \to \mathcal{U}$ be a measurable map into a measurable space $(\mathcal{U}, \Sigma_{\mathcal{U}})$. The join $\mathcal{F} \vee \sigma(U)$ is the smallest sigma-algebra $\mathcal{G} \subseteq \mathcal{E}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$-measurable.*

Proof. By definition,

$$\mathcal{F} \vee \sigma(U) = \sigma(\mathcal{F} \cup \sigma(U)).$$

Therefore $\mathcal{F} \subseteq \mathcal{F} \vee \sigma(U)$ and $\sigma(U) \subseteq \mathcal{F} \vee \sigma(U)$, so U is measurable with respect to $\mathcal{F} \vee \sigma(U)$.

Let $\mathcal{G}$ be any sigma-algebra such that $\mathcal{F} \subseteq \mathcal{G}$ and U is $\mathcal{G}$-measurable. The latter condition means $\sigma(U) \subseteq \mathcal{G}$. Hence

$$\mathcal{F} \cup \sigma(U) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(U)) \subseteq \mathcal{G}.$$

Thus $\mathcal{F} \vee \sigma(U) \subseteq \mathcal{G}$. □

Theorem O.3 (Contradiction requires distinction, formal version of Corollary F.6)**.** *Let $I : \Omega \to \mathcal{Z}$ be an information map and let $y : \Omega \to \mathsf{A}$ be a required output. Suppose $I(\omega) = I(\omega')$ but $y(\omega) \neq y(\omega')$. Then no predictor $f = g \circ I$ satisfies both*

$$f(\omega) = y(\omega), \qquad f(\omega') = y(\omega').$$

Proof. Since $f = g \circ I$ and $I(\omega) = I(\omega')$,

$$f(\omega) = f(\omega').$$

If both requirements held, then

$$y(\omega) = f(\omega) = f(\omega') = y(\omega'),$$

contradicting $y(\omega) \neq y(\omega')$. □

P Value of Distinction Proofs

The proofs below use standard tools from statistical decision theory, information theory, proper scoring rules, and conditional expectation: Bayes risk under constrained information, log-loss optimality of the conditional law, and the L^2 projection identity for conditional expectation [Blackwell, 1953, Berger, 1985, Cover and Thomas, 2006, Gneiting and Raftery, 2007, Billingsley, 1995, Kallenberg, 2002]. The derivations are included only to specialize these standard identities to the paper's notation.

Throughout the measurable-space statements, the action space A is equipped with a sigma-algebra Σ_{A}, actions are measurable maps into $(\mathsf{A}, \Sigma_{\mathsf{A}})$, and losses are measurable with finite expectation for the actions under consideration.

Theorem P.1 (Loss-relevant refinement, formal version of Theorem H.3)**.** *Let $\mathcal{F} \subseteq \mathcal{G} \subseteq \mathcal{E}$. Assume Bayes actions exist under $\mathcal{F}$ and $\mathcal{G}$, and that the $\mathcal{G}$-Bayes action is almost surely unique. Then*

$$R_L(\mathcal{F}) - R_L(\mathcal{G}) = 0$$

if and only if $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version.

Proof. If $a^\star_{\mathcal{G}}$ is $\mathcal{F}$-measurable, then it is feasible for $R_L(\mathcal{F})$, so

$$R_L(\mathcal{F}) \leq \mathbb{E}[L(Y, a^\star_{\mathcal{G}})] = R_L(\mathcal{G}).$$

The inclusion $\mathcal{F} \subseteq \mathcal{G}$ gives $R_L(\mathcal{G}) \leq R_L(\mathcal{F})$, so the risks are equal. Conversely, if the risks are equal, let $a^\star_{\mathcal{F}}$ be an $\mathcal{F}$-Bayes action. It is also $\mathcal{G}$-measurable and achieves $R_L(\mathcal{G})$. By almost sure uniqueness of $a^\star_{\mathcal{G}}$, $a^\star_{\mathcal{F}} = a^\star_{\mathcal{G}}$ almost surely. Hence $a^\star_{\mathcal{G}}$ has an $\mathcal{F}$-measurable version. □

Theorem P.2 (Bayes-risk monotonicity, formal version of Theorem H.2)**.** *Let Y be a target random variable, let A be an action space, and let L be an integrable loss. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

Proof. Define

$$\mathcal{D}_{\mathcal{F}} = \{a : \Omega \to \mathsf{A} : a \text{ is } \mathcal{F}\text{-measurable}\}, \qquad \mathcal{D}_{\mathcal{G}} = \{a : \Omega \to \mathsf{A} : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Since $\mathcal{F} \subseteq \mathcal{G}$,

$$\mathcal{D}_{\mathcal{F}} \subseteq \mathcal{D}_{\mathcal{G}}.$$

Therefore

$$\begin{aligned} R_L(\mathcal{G}) &= \inf_{a \in \mathcal{D}_{\mathcal{G}}} \mathbb{E}[L(Y, a)] \\ &\leq \inf_{a \in \mathcal{D}_{\mathcal{F}}} \mathbb{E}[L(Y, a)] \\ &= R_L(\mathcal{F}). \end{aligned}$$

□

Theorem P.3 (Log-loss value, formal version of Theorem H.4)**.** *Let Y be finite and let $\mathcal{F} \subseteq \mathcal{G}$. Under log loss, an admissible action is a measurable conditional distribution $q(\cdot)$ on the range of Y, and the loss is $-\log q(Y)$. Then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}) = \mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{G}) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right].$$

Proof. For any $\mathcal{F}$-measurable conditional distribution q, write $p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F})$. Then

$$\begin{aligned} \mathbb{E}[-\log q(Y) \mid \mathcal{F}] &= \sum_y p_{\mathcal{F}}(y)(-\log q(y)) \\ &= \sum_y p_{\mathcal{F}}(y)(-\log p_{\mathcal{F}}(y)) + \sum_y p_{\mathcal{F}}(y) \log \frac{p_{\mathcal{F}}(y)}{q(y)} \\ &= H(Y \mid \mathcal{F}) + D_{\mathrm{KL}}(p_{\mathcal{F}} \| q). \end{aligned}$$

Taking expectation and minimizing over q gives

$$R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F}).$$

Likewise,

$$R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G}).$$

By definition of conditional mutual information between Y and the additional information $\mathcal{G}$ given $\mathcal{F}$,

$$I(Y; \mathcal{G} \mid \mathcal{F}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}).$$

The conditional-KL form follows from the tower property:

$$\begin{aligned} &\mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{G}) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right] \\ &\quad = \mathbb{E}\left[\sum_y p_{\mathcal{G}}(y) \log p_{\mathcal{G}}(y) - \sum_y p_{\mathcal{G}}(y) \log p_{\mathcal{F}}(y)\right] \\ &\quad = -H(Y \mid \mathcal{G}) - \mathbb{E}\left[\sum_y p_{\mathcal{F}}(y) \log p_{\mathcal{F}}(y)\right] \\ &\quad = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}). \end{aligned}$$

Combining proves the claim. □

Corollary P.4 (Binary distinction value, formal version of Corollary H.5)**.** *Let $U \in \{0, 1\}$ and $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Under log loss,*

$$\begin{aligned} R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) &= I(Y; U \mid \mathcal{F}) \\ &= \mathbb{E}\left[\sum_{u \in \{0,1\}} \mathbb{P}(U = u \mid \mathcal{F}) D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}, U = u) \| \mathbb{P}(Y \in \cdot \mid \mathcal{F}))\right]. \end{aligned}$$

Proof. Apply Theorem P.3 with $\mathcal{G} = \mathcal{F} \vee \sigma(U)$. Conditioning on $\mathcal{F} \vee \sigma(U)$ is the same as conditioning on $\mathcal{F}$ and the binary value of U, yielding the displayed expansion over $u \in \{0, 1\}$. □

Theorem P.5 (Squared-loss value, formal version of Theorem H.6)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

Proof. The optimal $\mathcal{F}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, and the optimal $\mathcal{G}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{G}]$. Let

$$\varepsilon = Y - \mathbb{E}[Y \mid \mathcal{G}], \qquad \delta = \mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}].$$

Then

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = \varepsilon + \delta.$$

Since δ is $\mathcal{G}$-measurable and $\mathbb{E}[\varepsilon \mid \mathcal{G}] = 0$,

$$\mathbb{E}[\langle \varepsilon, \delta \rangle \mid \mathcal{G}] = \langle \mathbb{E}[\varepsilon \mid \mathcal{G}], \delta \rangle = 0.$$

Thus

$$\begin{aligned}
\mathbb{E}\,\|Y - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2 &= \mathbb{E}\,\|\varepsilon + \delta\|_2^2 \\
&= \mathbb{E}\,\|\varepsilon\|_2^2 + \mathbb{E}\,\|\delta\|_2^2 \\
&= R_2(\mathcal{G}) + \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].
\end{aligned}$$

Rearranging proves the identity. □

Theorem P.6 (Minimal loss-sufficient refinement, formal version of Theorem H.7)**.** *Let $\mathcal{F} \subseteq \mathcal{G}$ and assume the $\mathcal{G}$-Bayes action $a_{\mathcal{G}}^\star : \Omega \to \mathsf{A}$ exists as a $(\mathcal{G}, \Sigma_{\mathsf{A}})$-measurable map and is almost surely unique. Also assume the Bayes infimum is attained for every intermediate sigma-algebra $\mathcal{H}$ with $\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}$ that is considered below. Up to null sets, the smallest sigma-algebra $\mathcal{H}$ satisfying*

$$\mathcal{F} \subseteq \mathcal{H} \subseteq \mathcal{G}, \qquad R_L(\mathcal{H}) = R_L(\mathcal{G})$$

is $\mathcal{F} \vee \sigma(a_{\mathcal{G}}^\star)$. For finite Y under log loss, the corresponding smallest sufficient refinement is $\mathcal{F} \vee \sigma(p_{\mathcal{G}})$, where $p_{\mathcal{G}}(\cdot) = \mathbb{P}(Y \in \cdot \mid \mathcal{G})$.

Proof. The action $a_{\mathcal{G}}^\star$ is measurable with respect to $\mathcal{F} \vee \sigma(a_{\mathcal{G}}^\star)$, so this refinement attains $R_L(\mathcal{G})$. Let $\mathcal{H}$ be any intermediate sigma-algebra with $R_L(\mathcal{H}) = R_L(\mathcal{G})$. Since $\mathcal{H} \subseteq \mathcal{G}$ and the $\mathcal{G}$-Bayes action is almost surely unique, Theorem P.1 implies that $a_{\mathcal{G}}^\star$ has an $\mathcal{H}$-measurable version. Therefore $\sigma(a_{\mathcal{G}}^\star) \subseteq \mathcal{H}$ up to null sets, and because $\mathcal{F} \subseteq \mathcal{H}$,

$$\mathcal{F} \vee \sigma(a_{\mathcal{G}}^\star) \subseteq \mathcal{H}$$

up to null sets. Under log loss, the Bayes action is the conditional law $p_{\mathcal{G}}$, giving the stated form. □

Proposition P.7 (Finite squared-loss split identity)**.** *Let $B = B_0 \cup B_1$ be a disjoint split of a finite sample set. With empirical means $\hat{\mu}_B, \hat{\mu}_{B_0}, \hat{\mu}_{B_1}$ and* SSE *as in Section H,*

$$\mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1) = |B_0|\,\|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1|\,\|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2.$$

Proof. For each child B_b,

$$\begin{aligned}
\sum_{j \in B_b} \|y_j - \hat{\mu}_B\|_2^2 &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b} + \hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2 \\
&= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b}\|_2^2 + |B_b|\,\|\hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2,
\end{aligned}$$

because

$$\sum_{j \in B_b} (y_j - \hat{\mu}_{B_b}) = 0.$$

Summing over $b \in \{0, 1\}$ gives the result. □

Q CGA Algorithm and Projection-Bridge Proofs

This appendix combines elementary finite graph/partition arguments with standard concentration and linear-algebra tools: Hoeffding sample-split validation bounds, adaptive-holdout caveats, nearest-neighbor indexing costs, Moore-Penrose projections, and ridge/Tikhonov least squares [Hoeffding, 1963, Devroye et al., 1996, Dwork et al., 2015b,a, Indyk and Motwani, 1998, Johnson et al., 2019, Penrose, 1955, Tikhonov and Arsenin, 1977, Hoerl and Kennard, 1970].

Theorem Q.1 (Anchor edge separation, formal version of Theorem J.5)**.** *Let $a_i, a_j \in \mathbb{R}^d$ with $a_i \neq a_j$. Define*

$$u = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b = \frac{1}{2}u^\top(a_i + a_j).$$

Then

$$u^\top a_i - b = \frac{1}{2}\|a_i - a_j\|_2, \qquad u^\top a_j - b = -\frac{1}{2}\|a_i - a_j\|_2.$$

Proof. Compute

$$\begin{aligned} u^\top a_i - b &= u^\top a_i - \frac{1}{2}u^\top a_i - \frac{1}{2}u^\top a_j \\ &= \frac{1}{2}u^\top(a_i - a_j) \\ &= \frac{1}{2}\|a_i - a_j\|_2. \end{aligned}$$

Similarly,

$$\begin{aligned} u^\top a_j - b &= u^\top a_j - \frac{1}{2}u^\top a_i - \frac{1}{2}u^\top a_j \\ &= -\frac{1}{2}u^\top(a_i - a_j) \\ &= -\frac{1}{2}\|a_i - a_j\|_2. \end{aligned}$$

□

The construction is used only for separated anchors. In implementations, proposed edges with $\|a_i - a_j\|_2 \leq \eta_{\text{anchor}}$ are skipped or replaced by a fallback distinction, so the normalized direction is never formed from duplicate or numerically indistinguishable anchors.

Theorem Q.2 (CGA graph generates a finite edge-skeleton information structure, formal version of Theorem J.6)**.** *Let S be a finite set of binary generators $g_s : \mathsf{H} \to \{0, 1\}$. Define*

$$\sigma_S(h) = \big(g_s(h)\big)_{s \in S}.$$

The relation $h \sim_S h'$ iff $\sigma_S(h) = \sigma_S(h')$ is an equivalence relation. Its nonempty equivalence classes form a finite edge-skeleton partition $\mathcal{P}_S^G$ of H. If a new generator g is added, then

$$\mathcal{P}_{S \cup \{g\}}^G = \mathcal{P}_S^G \vee \sigma(g),$$

where the right side denotes cutting each atom of $\mathcal{P}_S^G$ by $g^{-1}(1)$ and $g^{-1}(0)$ and keeping nonempty pieces.

Proof. Equality of signatures is reflexive, symmetric, and transitive, so $\sim_S$ is an equivalence relation. Equivalence classes are disjoint, nonempty, and cover H, hence they form a partition. Since S is finite, there are at most $2^{|S|}$ signatures, so the partition is finite.

For the update, two points have the same signature under $S \cup \{g\}$ exactly when they have the same old S-signature and the same value of g. Therefore each new equivalence class is an old atom intersected with either $g^{-1}(1)$ or $g^{-1}(0)$, with empty intersections discarded. This is precisely $\mathcal{P}_S^G \vee \sigma(g)$. □

Theorem Q.3 (Anchor-based hard routing is graph-plus-anchor-order measurable, formal version of Theorem J.7)**.** *Let E_t be finite and let $a_1, \ldots, a_{N_t} \in \mathbb{R}^d$ be finite frozen anchors with deterministic tie-breaking. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be generated by all pairwise anchor-distance comparison events*

$$\{h : \|h - a_i\|_2^2 \leq \|h - a_j\|_2^2\}.$$

The exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.

Proof. For each concept j, eligibility is an intersection of finitely many edge-bit events of the form

$$\{h : g_e(h) = 1\} \qquad \text{or} \qquad \{h : g_e(h) = 0\},$$

depending on j's recorded side of edge e. Hence the eligible set for j is $\mathcal{F}_t^{\text{edge}}$-measurable.

The anchor-order sigma-algebra determines the complete order of all anchors under the fixed tie-breaking rule. For any subset $R \subseteq \{1, \ldots, N_t\}$, the event that concept j is the nearest anchor in R is a finite intersection of pairwise anchor-comparison events, hence belongs to $\mathcal{F}_t^{\text{anchor-order}}$. The selected hard-route index is obtained by combining finitely many eligibility events with these nearest-in-subset events. Therefore the inverse image of each selected index is a finite union of intersections of sets from $\mathcal{F}_t^{\text{edge}}$ and $\mathcal{F}_t^{\text{anchor-order}}$, and belongs to their join. □

Theorem Q.4 (Soft routing approaches hard routing, formal version of Theorem J.9)**.** *Let N be a finite candidate set and let*

$$\alpha_j = \frac{\exp(-d_j/T)\chi_j}{\sum_{r \in N} \exp(-d_r/T)\chi_r}.$$

If $j^\star_{\text{hard}} \notin N$, where $j^\star_{\text{hard}}$ is the exact hard-route selected concept, then this sparse soft read gives no guarantee for the omitted hard-route concept. Otherwise, assume $j^\star = j^\star_{\text{hard}} \in N$. For an eligible competitor set $E \subseteq N \setminus \{j^\star\}$, if $d_j - d_{j^\star} \geq \delta > 0$ and $\chi_j/\chi_{j^\star} \leq C_E < \infty$ for all $j \in E$, then

$$\sum_{j \in E} \alpha_j \leq C_E \, |E| \exp(-\delta/T).$$

Moreover, suppose $\log \chi_j$ contains a term

$$\frac{\beta_e}{\sqrt{D_j}} \log \operatorname{sigmoid}\left(\frac{m_j}{\tau_e}\right)$$

with $\beta_e > 0$ and finite $D_j \geq 1$, and candidate j violates that edge with $m_j \leq -\delta_e < 0$. Then $\chi_j \leq \exp(-\beta_e \delta_e / (\sqrt{D_j}\tau_e))$.

For a wrong-side set $W \subseteq N$, suppose every $j \in W$ violates some incident structural edge whose weight is at least $\beta_{\min} > 0$, whose signed margin is at most $-\delta_e$, and whose incident degree is at most $D_{\max}$. If at least one eligible reference candidate r has $\chi_r \geq \chi_{\min} > 0$, and the distance score ratio between every $j \in W$ and r is bounded by C, then the total wrong-side routing mass is at most

$$\frac{C\,|N|}{\chi_{\min}} \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

More generally, if

$$\Delta_d = \max_{j \in W} (d_r - d_j)_+,$$

then the same mass is at most

$$\frac{|N|}{\chi_{\min}} \exp\left(\frac{\Delta_d}{T} - \frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right),$$

and therefore vanishes along any joint limit satisfying

$$\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e} - \frac{\Delta_d}{T} \to +\infty.$$

Consequently, if $N \setminus \{j^\star\}$ is covered by eligible competitors satisfying the distance-margin and compatibility-ratio bounds and wrong-side competitors satisfying either wrong-side bound above, then $\alpha_{j^\star} \to 1$ under the corresponding temperature limit.

Proof. If $j^\star_{\text{hard}} \notin N$, then the soft normalization assigns weight only to candidates in N and the omitted hard-route concept has no route to receive weight. This is the candidate-recall caveat.

For the eligible-competitor statement, the denominator is at least $\exp(-d_{j^\star}/T)\chi_{j^\star}$, so

$$\begin{aligned}\sum_{j\in E}\alpha_j &\le \sum_{j\in E}\frac{\exp(-d_j/T)\chi_j}{\exp(-d_{j^\star}/T)\chi_{j^\star}}\\ &\le C_E\sum_{j\in E}\exp(-(d_j-d_{j^\star})/T) \le C_E\,|E|\exp(-\delta/T).\end{aligned}$$

For an edge violation, set $a=\delta_e/\tau_e>0$. Since

$$\text{sigmoid}(-a)=\frac{1}{1+\exp(a)}\le\exp(-a),$$

the corresponding log-compatibility contribution is at most $-\beta_e\delta_e/(\sqrt{D_j}\tau_e)$, and therefore $\chi_j \le \exp(-\beta_e\delta_e/(\sqrt{D_j}\tau_e))$. If W is the wrong-side set and r is an eligible candidate whose compatibility satisfies $\chi_r \ge \chi_{\min}$, the uniform wrong-side assumptions give, for every $j\in W$,

$$\chi_j \le \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

Combining this with the assumed distance-score ratio bound gives

$$\begin{aligned}\sum_{j\in W}\alpha_j &\le \sum_{j\in W}\frac{C}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right)\\ &\le \frac{C\,|N|}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).\end{aligned}$$

Without a uniform distance-score ratio, use

$$\frac{\exp(-d_j/T)}{\exp(-d_r/T)}=\exp\left(\frac{d_r-d_j}{T}\right)\le\exp(\Delta_d/T),$$

which gives the joint-limit bound and the stated convergence condition. □

Theorem Q.5 (Refine-or-repair objective descent, formal version of Theorem J.11)**.** *Let*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S})=\widehat{R}(\mathcal{S};\mathcal{B}_t^{\text{score}})+\lambda_{\text{concept}}N_{\text{concept}}(\mathcal{S})+\lambda_{\text{edge}}\,|E(\mathcal{S})|$$

with nonnegative penalties on a fixed scoring buffer, where λ_{concept} includes the anchored concept and the anchor-order bookkeeping induced by creating it. If a repair step is committed only after the guarded test

$$\widehat{R}(\mathcal{S}_{t+1};\mathcal{B}_t^{\text{score}})\le\widehat{R}(\mathcal{S}_t;\mathcal{B}_t^{\text{score}}),$$

and leaves anchors, edges, and concept count unchanged, then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement creates one concept and actual edge set E_{new} and is accepted only when

$$\widehat{\Delta}_{\text{refine}}>\max\{\widehat{\Delta}_{\text{repair}},0\}+\tau+4\epsilon_m,$$

where $\widehat{\Delta}_{\text{repair}}\ge 0$ is the empirical repair gain and

$$\widehat{\Delta}_{\text{refine}}=\widehat{R}(\mathcal{S}_t;\mathcal{B}_t^{\text{score}})-\widehat{R}(\mathcal{S}_{t+1};\mathcal{B}_t^{\text{score}})-\lambda_{\text{concept}}-\lambda_{\text{edge}}\,|E_{\text{new}}|\,,$$

then

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1})<J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t)-\widehat{\Delta}_{\text{repair}}-\tau-4\epsilon_m.$$

In particular, $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\tau+4\epsilon_m$.

Proof. For repair, N_{concept} and $|E|$ are unchanged. The empirical-risk term on $\mathcal{B}_t^{\text{score}}$ is no larger by the guard, so $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. The identity candidate is always feasible, so the guarded rule can decline harmful centroid or memory movement instead of committing it.

For refinement,

$$\begin{aligned} J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) &= \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} |E_{\text{new}}| \\ &= \widehat{\Delta}_{\text{refine}} \\ &> \widehat{\Delta}_{\text{repair}} + \tau + 4\epsilon_m. \end{aligned}$$

Rearranging gives the claim. □

The theorem is local to the fixed scoring buffer used in its statement. If the scoring buffer changes over time, these inequalities alone do not define a common global descent potential.

Theorem Q.6 (Statistically safe adaptive refinement, formal version of Theorem J.12)**.** *Assume losses lie in* $[0, 1]$*. Let* $(\mathfrak{H}_t)$ *be the filtration containing the past stream, current CGA state, and all proposal-buffer information available before scoring at time* t*. Let* $\mathcal{C}_t$ *be a finite candidate class predictable with respect to* $\mathfrak{H}_t$*, and let* $\mathcal{B}_t^{\text{score}}$ *contain* m_t *fresh held-out samples, or samples not previously used to choose proposals or update the state, independent of* $\mathfrak{H}_t$ *conditional on the current scoring distribution. For candidate* g*, let* $\widehat{G}_t(g)$ *be the empirical raw risk decrease on* $\mathcal{B}_t^{\text{score}}$ *before complexity cost and let* $G_t(g)$ *be the corresponding population raw gain. Define*

$$\epsilon_t = \sqrt{\frac{\log(4t^2(|\mathcal{C}_t| + 1)/\delta)}{2m_t}}.$$

Then, with probability at least $1 - \delta$*, every time* t *and candidate* $g \in \mathcal{C}_t$ *satisfy*

$$\left| \widehat{G}_t(g) - G_t(g) \right| \leq 2\epsilon_t.$$

Consequently, if

$$\widehat{G}_t(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_t,$$

then on the same event

$$G_t(g) > \lambda \operatorname{cost}(g) + \tau.$$

Proof. Condition on $\mathfrak{H}_t$. The candidate class is then fixed, and the scoring buffer is independent of the proposal mechanism and prior state-selection decisions by assumption. Apply Hoeffding's inequality to the empirical risk of the old state and to the empirical risk of each candidate-refined state. For any fixed state in this finite collection,

$$\mathbb{P}\left(\left| \widehat{R} - R \right| > \epsilon \right) \leq 2 \exp(-2m\epsilon^2).$$

With $m = m_t$ and $\epsilon = \epsilon_t$, the failure probability at time t is at most $\delta/(2t^2)$ after a union bound over the old state and the $|\mathcal{C}_t|$ candidate states. Since $\sum_{t \geq 1} t^{-2} < 2$, the displayed constant $4t^2$ gives total failure probability at most δ over all times. On this event, each raw gain is a difference of two risks, so empirical and population raw gain differ by at most $2\epsilon_t$. The implication follows by subtracting this deviation from the empirical raw-gain lower bound. □

Corollary Q.7 (Bounded language-model refinement score, formal version of Corollary J.13)**.** *For language modeling, let*

$$\ell_{\text{clip}}(x_{\leq t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\leq t}), L_{\max}\}$$

for a fixed $L_{\max} > 0$*. If the scoring buffer, empirical raw gain, and population raw gain in Theorem Q.6 are computed using* ℓ_{clip}*, then the theorem applies to language-model refinement decisions made with this clipped score. In particular, an accepted refinement has positive clipped population gain after paying complexity cost and margin* τ *with probability at least* $1 - \delta$*.*

Proof. The clipped score satisfies $0 \le \ell_{\text{clip}} \le 1$. Substituting this bounded loss into Theorem Q.6 gives the claim directly. The statement is only about the clipped score; raw negative log likelihood is unbounded and would require a separate concentration assumption. □

Theorem Q.8 (Anchor-stable CGA information structure, formal version of Theorem J.14)**.** *Let*

$$\mathcal{F}_t^{\text{CGA}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\} : 1 \le i, j \le N_t\Big).$$

Suppose every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is computed from frozen endpoint anchors a_i, a_j. Any repair step that changes only centroids c_j, local memories M_j, and counts n_j, while keeping anchors and edges fixed, leaves $\mathcal{F}_t^{\text{CGA}}$ unchanged. If a refinement adds a new anchor and new edge generators, then $\mathcal{F}_{t+1}^{\text{CGA}}$ is $\mathcal{F}_t^{\text{CGA}}$ joined with the new anchor-order events and the new edge generators.

Proof. For an edge $e = (i, j)$, both

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j)$$

are deterministic functions of the frozen anchors. A repair step does not change these anchors, so it does not change u_e, b_e, or g_e. It also does not change any anchor-distance comparison event because all anchors are fixed. Hence both sigma-algebras in the join defining $\mathcal{F}_t^{\text{CGA}}$ are unchanged.

If a refinement adds anchor a_{new} and edge generators E_{new}, the new edge sigma-algebra is $\sigma(g_e : e \in E_t) \vee \sigma(g_e : e \in E_{\text{new}})$, and the new anchor-order sigma-algebra is the old anchor-order sigma-algebra joined with the pairwise comparisons involving a_{new}. This is exactly the stated join. □

Corollary Q.9 (Finite number of accepted refinements, formal version of Corollary J.15)**.** *Let Φ be a common potential with $\Phi(\mathcal{S}) \ge 0$ for all reachable states. If each accepted refinement decreases Φ by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$.*

Proof. After m accepted refinements,

$$\Phi(\mathcal{S}_m) \le \Phi(\mathcal{S}_0) - m\tau.$$

Since $\Phi(\mathcal{S}_m) \ge 0$, we have $m \le \Phi(\mathcal{S}_0)/\tau$. □

Proposition Q.10 (Trees as generalized hard-route CGAs, formal version of Proposition N.1)**.** *Let $\mathcal{T}$ be a binary decision tree whose internal nodes compute binary distinctions $g_\nu(h) \in \{0, 1\}$ and whose leaves have predictors M_ℓ. Construct a generalized hard-route CGA with one concept c_ℓ and memory M_ℓ for each leaf ℓ, where the generalized model permits arbitrary finite binary eligibility constraints in addition to pairwise distinction edges. For each leaf, add hard eligibility constraints equal to the signed distinctions along the root-to-leaf path. If the generalized hard route selects the unique eligible leaf concept, then its prediction equals the tree prediction for every h.*

Proof. For any input h, the tree follows exactly one root-to-leaf path. The leaf reached by the tree is the unique leaf whose path constraints are all satisfied by h. By construction, the corresponding generalized hard-route concept has exactly those eligibility constraints, and all other leaf concepts violate at least one path constraint at the first node where their path differs from the tree path. Therefore the generalized hard route selects the same leaf and applies the same memory M_ℓ. □

Theorem Q.11 (CGA data-structure bound, formal version of Theorem N.2)**.** *Assume CGA has N_t concepts in $\mathbb{R}^d$, exact centroid and anchor retrieval, candidate set $N(h)$, selected incident edge-gate count $Q(h) = \sum_{j \in N(h)} |\text{Inc}_t(j)|$, and local-memory evaluation cost C_{local}. Then one prediction costs $O(N_t d + Q(h)d + |N(h)|\, d + C_{\text{local}})$ for dense linear edge gates. One observation that repairs a selected concept costs this plus the guarded local repair scoring cost. One observation that creates a new anchor-centroid pair and q guarded distinction edges costs an additional $O(d + qd)$ plus anchor-order query bookkeeping. If each concept stores an anchor, a centroid, and a sparse categorical memory with average support $\bar{s}$, and each edge stores a linear distinction in $\mathbb{R}^d$, then the runtime state uses $O(N_t(2d + \bar{s}) + |E_t|\, d)$ memory.*

Proof. A distance computation between $h \in \mathbb{R}^d$ and one concept center costs $O(d)$. Exact centroid and anchor retrieval over N_t concepts therefore costs

$$O(N_t d).$$

Graph-neighbor expansion has cost proportional to the incident lists touched and is lower order relative to dense edge-gate evaluation. After the candidate set $N(h)$ is selected, evaluating the incident linear edge gates costs $O(Q(h)d)$. Computing sparse routing weights, distances within the selected set, and evaluating selected local memories costs

$$O(|N(h)|\, d + C_{\text{local}}).$$

Thus prediction costs

$$O(N_t d + Q(h)d + |N(h)|\, d + C_{\text{local}}).$$

Observation begins with the same retrieval computation. If it repairs a selected concept, it adds the cost of evaluating and scoring the proposed local repair candidates on the scoring buffer. If it creates a new anchor-centroid pair, storing the pair costs $O(d)$. For each new edge, computing and storing the linear distinction vector costs $O(d)$, so q edges cost

$$O(qd).$$

The total additional vector-update cost for refinement is therefore $O(d + qd)$, plus whatever indexing or comparison bookkeeping is used to support anchor-order queries.

For memory, each concept stores $a_j, c_j \in \mathbb{R}^d$ and a sparse categorical memory of average support $\bar{s}$, giving

$$O(N_t(2d + \bar{s})).$$

Each edge stores one distinction vector and threshold, giving

$$O(|E_t|\, d).$$

Adding the two terms proves the claimed bound. □

Theorem Q.12 (CGA layer cost, formal version of Theorem N.3)**.** *Consider a CGA adapter extension with hidden dimension d, query dimension d_q, local rank r, candidate count $|N|$, selected incident edge-gate count Q_N, and concept-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N d_q + |N|\, dr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N d_q + |N|\, (d_q + dr)).$$

For a block of length L, backpropagation through the slow parameters does not require storing the full CGA state history $\mathcal{S}_1, \ldots, \mathcal{S}_L$.

Proof. For a token hidden state $h \in \mathbb{R}^d$, the query computation

$$q = Qh$$

costs $O(dd_q)$. The retrieval cost is $C_{\text{ret}}(N_t, d_q)$ by definition. Evaluating selected incident dense edge gates costs $O(Q_N d_q)$.

For a selected concept j, the rank-r adapter read is

$$W_j K h.$$

Multiplying one selected $W_j \in \mathbb{R}^{d \times r}$ by a rank-r key costs $O(dr)$. Therefore $|N|$ selected adapters cost

$$O(|N|\, dr).$$

The routing weights add lower-order top-k normalization cost, absorbed into the same bound. This proves the read overhead.

The write path first computes the same query, retrieval, and edge gates. It then updates selected concept statistics and selected local memories. Concept bookkeeping costs $O(d_q)$ per selected concept. A rank-one local repair of a rank-r or low-rank sufficient-statistic memory costs $O(dr)$ per selected memory. Hence the write overhead is

$$O(dd_q + C_{\mathrm{ret}}(N_t, d_q) + Q_N d_q + |N|\,(d_q + dr)).$$

Finally, the update rule is applied as

$$\mathcal{S}_{t+1} = \mathrm{stopgrad}(\mathrm{CGAObserve}(\mathrm{stopgrad}(\mathcal{S}_t), \mathrm{stopgrad}(h_t), x_{t+1}))\,.$$

Thus the computational graph for the loss contains the selected read computation, but not the chain of state-update maps from $\mathcal{S}_1$ through $\mathcal{S}_L$ or the write-time hidden states used to mutate the fast state. Therefore full CGA state history is not required for backpropagation through the slow parameters. □

Theorem Q.13 (One-hot memory is partition-measurable, formal version of Theorem M.1)**.** *Let $\mathcal{P} = \{B_1, \dots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}}(\omega) = e_i$ for $\omega \in B_i$. For every $W \in \mathbb{R}^{m\times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is equal to f_W for some $W \in \mathbb{R}^{m\times n}$.*

Proof. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

Thus

$$f_W(\omega) = We_i = f_W(\omega'),$$

so f_W is constant on each atom and is $\mathcal{P}$-measurable.

Conversely, if f is $\mathcal{P}$-measurable, it is constant on each B_i. Let w_i be this constant value on B_i, and define

$$W = [w_1, \dots, w_n].$$

Then for $\omega \in B_i$,

$$W\phi_{\mathcal{P}}(\omega) = We_i = w_i = f(\omega).$$

□

Theorem Q.14 (CPM conflict equals missing distinction, formal version of Theorem M.2)**.** *Let $\mathcal{P} = \{B_1, \dots, B_n\}$ and let $\omega, \omega' \in B_i$. Suppose $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If a memory W has already committed $We_i = v(\omega)$, then the evidence $We_i = v(\omega')$ has innovation $z = 0$ relative to the old key span and residual $e \neq 0$.*

Proof. Here the stored-key matrix K is oriented with stored keys as columns, so $KK^\dagger$ is the orthogonal projector onto their column span. Since $\omega, \omega' \in B_i$,

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

The old key span contains e_i. Therefore the innovation of the new key $k = e_i$ relative to the old key matrix $K = e_i$ is

$$z = (I - KK^\dagger)k = (I - e_i e_i^\top)e_i = 0.$$

The new residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega).$$

Since $v(\omega') \neq v(\omega)$, we have $e \neq 0$. □

NeurIPS Paper Checklist

This file is retained only for the conference-template branch of `main.tex`.

- **Claims.** The abstract and introduction state the paper's scope as a method proposal: CGA replaces token-indexed attention memory with a fixed bank of runtime concept slots.
- **Limitations.** Section 8 states the main limitations, including kernel optimization, retention tuning, concept-count selection, and the fact that 180M–440M LM results require completed training and latency reporting before any large-scale win can be claimed.
- **Theory.** Section 5 gives the state definition, read/write equations, state-size accounting, and backpropagation boundary for fixed-bank CGA.
- **Experiments.** Section 6 centers on MQAR as the controlled recall benchmark and a PoST-style CGA-LM pretraining/evaluation protocol as the large-scale sequence-modeling test.